

DATA MINING

THE DATA MINING PIPELINE

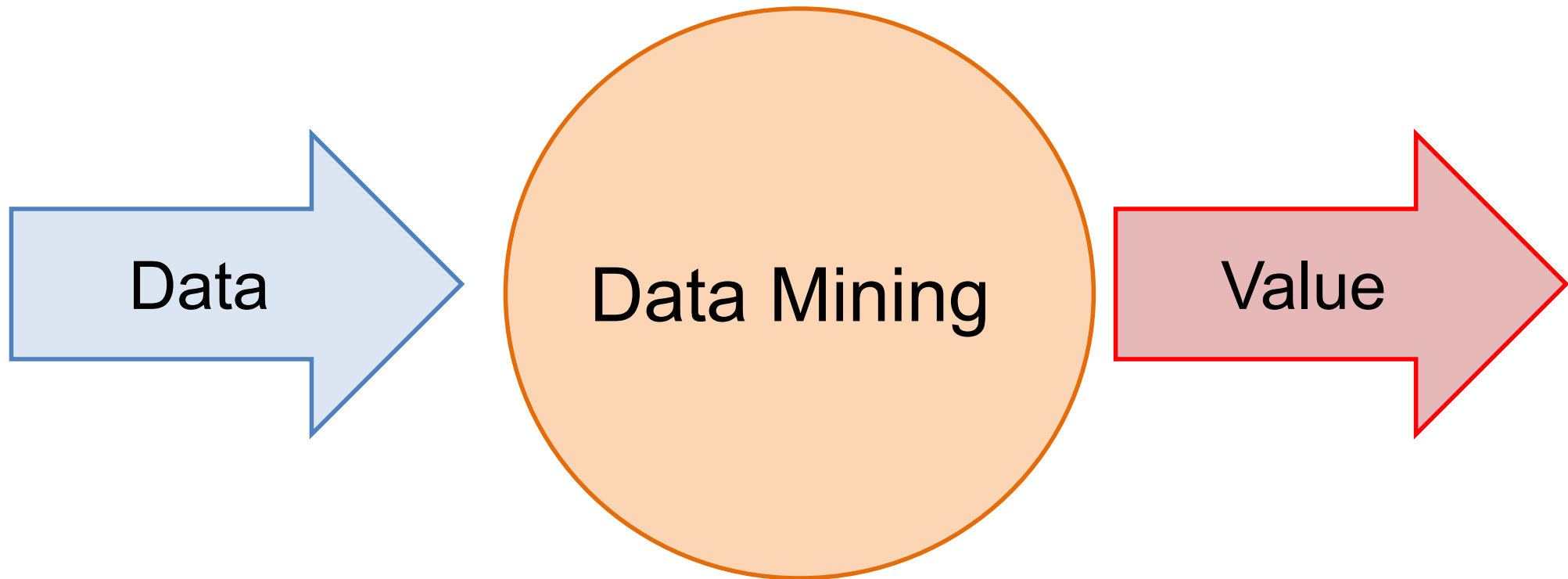
What is data?

The data mining pipeline: collection, preprocessing, mining, and post-processing

Sampling, feature extraction and normalization

What is data mining again?

- “Data Mining is the study of **collecting, processing, analyzing, and gaining useful insights** from data” – Charu Aggarwal



- Essentially, anything that has to do with data is data mining

DATA

What is Data?

- Collection of data **objects** and their **attributes**
- An attribute is a property or characteristic of an object
 - Examples: name, date of birth, height, occupation.
 - Attribute is also known as **variable**, **field**, **characteristic**, or **feature**
- For each object the attributes take some **values**.
- The collection of **attribute-value pairs** describes a specific object
 - Object is also known as **record**, **point**, **case**, **sample**, **entity**, or **instance**

Attributes

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Objects

Size (n): Number of objects

Dimensionality (d): Number of attributes

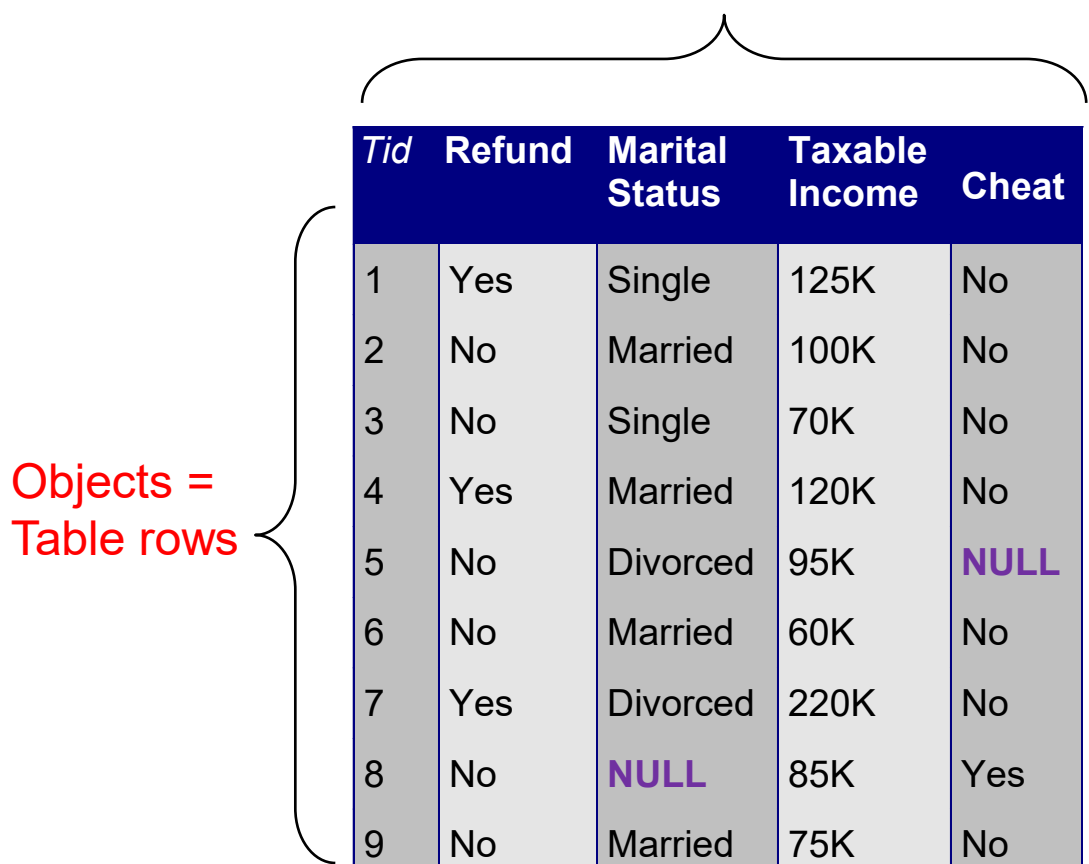
Density/Sparsity: Number of populated object-attribute pairs

Relational data

- The term comes from **DataBases**, where we assume data is stored in a **relational table** with a fixed schema (fixed set of attributes)
 - In Databases, it is usually assumed that the table is **dense** (few null values)
- There are a lot of data in this form
 - E.g., census data
- There are also a lot of data which do not fit well in this form
 - **Sparse** data: Many missing values
 - Not easy to define a fixed schema

Example of a relational table

Attributes = Table columns



<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	NULL
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	NULL	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Objects =
Table rows

Types of Attributes

- There are different types of attributes
 - **Numeric**
 - Examples: dates, temperature, time, length, value, count.
 - **Discrete** (counts) vs **Continuous** (temperature)
 - Special case: **Binary/Boolean** attributes (yes/no, exists/not exists)
 - **Categorical**
 - Examples: eye color, zip codes, strings, rankings (e.g, good, fair, bad), height in {tall, medium, short}
 - **Nominal** (no order or comparison) vs **Ordinal** (order but not comparable)

Numeric Relational Data

- If data objects have the same **fixed set** of **numeric attributes**, then the data objects can be thought of as **points/vectors** in a multi-dimensional space, where each **dimension** represents a distinct attribute
- Such data set can be represented by an **n-by-d data matrix**, where there are **n** rows, one for each object, and **d** columns, one for each attribute

	Temperature	Humidity	Pressure
O1	30	0.8	90
O2	32	0.5	80
O3	24	0.3	95

30	0.8	90
32	0.5	80
24	0.3	95

Numeric data from embeddings

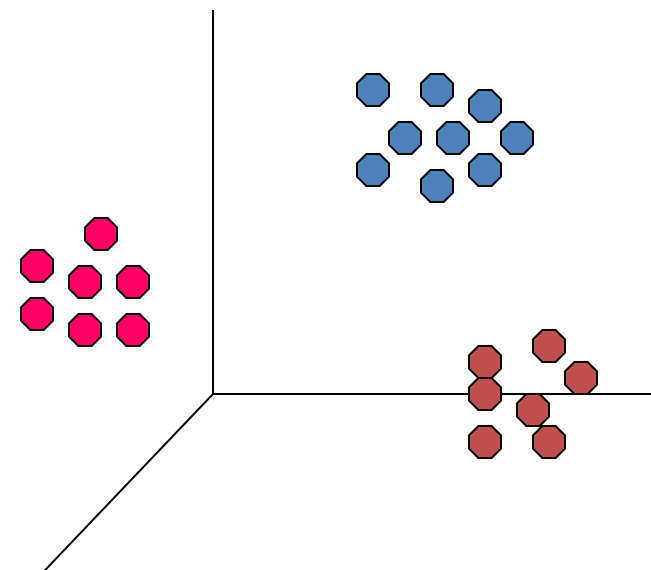
- **Embeddings** define a **representation** of the data objects in a high-dimensional vector space
 - **Representation learning**: Learn representations that preserve/highlight some property of the data, typically similarity
- In this numerical data, each dimension corresponds to a feature, but we do not know what this feature means
- The resulting **data matrix** is very **dense**, with **high dimension**.

Vector Databases

- This is a special case of numerical data, where each object is a **multidimensional numerical vector**.
- Different from before, the vectors do not store the attribute values for the object, but rather the **embedding** of the object as it is produced by some Machine Learning algorithm.
- Embedding vectors are fully **dense** and have no clear interpretation.
- They have been shown to be remarkably successful in capture **semantic relationships** between objects.

Numeric data

- Thinking of numeric data as **points** or **vectors** is very convenient
- For **small dimensions** we can **plot** the data
- We can use **geometric analogues** to define concepts like **distance** or **similarity**
- We can use **linear algebra** to process the **data matrix**
- We will often talk about points or vectors



Categorical Relational Data

- Data that consists of a collection of records, each of which consists of a **fixed set** of **categorical** attributes

ID Number	Zip Code	Marital Status	Income Bracket
1129842	45221	Single	High
2342345	45223	Married	Low
1234542	45221	Divorced	High
1243535	45224	Single	Medium

Mixed Relational Data

- Data that consists of a collection of records, each of which consists of a **fixed set** of both **numeric** and **categorical** attributes

ID Number	Zip Code	Age	Marital Status	Income	Income Bracket
1129842	45221	55	Single	250000	High
2342345	45223	25	Married	30000	Low
1234542	45221	45	Divorced	200000	High
1243535	45224	43	Single	150000	Medium

Mixed Relational Data

- Data that consists of a collection of records, each of which consists of a **fixed set** of both **numeric** and **categorical** attributes

ID Number	Zip Code	Age	Marital Status	Income	Income Bracket	Refund
1129842	45221	55	Single	250000	High	No
2342345	45223	25	Married	30000	Low	Yes
1234542	45221	45	Divorced	200000	High	No
1243535	45224	43	Single	150000	Medium	No

Mixed Relational Data

- Data that consists of a collection of records, each of which consists of a **fixed set** of both **numeric** and **categorical** attributes

Takes numerical values but it is actually categorical

ID Number	Zip Code	Age	Marital Status	Income	Income Bracket	Refund
1129842	45221	55	Single	250000	High	0
2342345	45223	25	Married	30000	Low	1
1234542	45221	45	Divorced	200000	High	0
1243535	45224	43	Single	150000	Medium	0

Boolean attributes can be thought as both numeric and categorical

When appearing together with other attributes they make more sense as **categorical**

They are often represented as numeric though

Mixed Relational Data

- Sometimes it is convenient to represent **categorical** attributes as **boolean**.
 - Add a Boolean attribute for each possible value of the attribute

ID	Zip 45221	Zip 45223	Zip 45224	Age	Single	Married	Divorced	Income	Refund
1129842	1	0	0	55	0	0	0	250000	0
2342345	0	1	0	25	0	1	0	30000	1
1234542	1	0	0	45	0	0	1	200000	0
1243535	0	0	1	43	0	0	0	150000	0

We can now view the whole vector as **numeric**

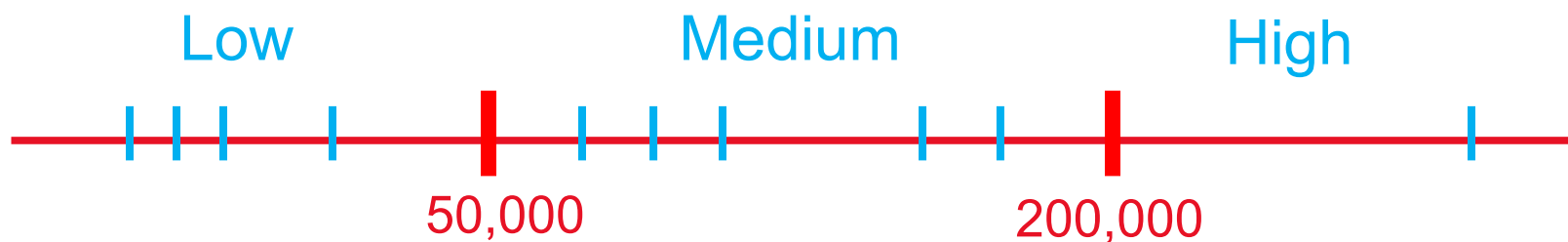
Mixed Relational Data

- Sometimes it is convenient to represent **numerical** attributes as **categorical**.
 - Group the values of the numerical attributes into **bins**

ID Number	Zip Code	Age	Marital Status	Income	Income Bracket	Refund
1129842	45221	50s	Single	High	High	0
2342345	45223	20s	Married	Low	Low	1
1234542	45221	40s	Divorced	High	High	0
1243535	45224	40s	Single	Medium	Medium	0

Binning

- Idea: split the range of the domain of the numerical attribute into bins (intervals).
- Every bucket defines a categorical value

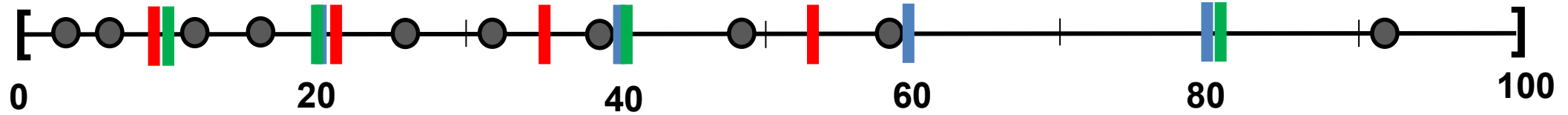


- How do we decide the number of bins?
 - Depends on the granularity of the data that we want
 - Sometimes domain knowledge is also used

Bucketization

- How do we decide the size of the bucket?
 - Depends on the data and our application
- **Equi-width** bins: All bins have the same size
 - Example: split time into decades
 - Problem: some bins may be very sparse or empty
- **Equi-size (depth)** bins: Select the bins so that they all contain the same number of elements
 - This splits data into **quantiles**: top-10%, second 10% etc
 - Some bins may be very small
- **Equi-log** bins: $\log end - \log start$ is constant
 - The size of the previous bin is a fraction of the current one
 - Better for skewed distributions
- **Optimized** bins: Use a 1-dimensional clustering algorithm to create the bins

Example



Blue: Equi-width [20,40,60,80]

Red: Equi-depth (2 points per bin)

Green: Equi-log ($\frac{end}{start} = 2$)

Physical data storage

- Stored in a **Relational Database**
 - Assumes a strict **schema** and relatively **dense** data (few missing/Null values)
- **Tab or Comma separated** files (TSV/CSV), **Excel** sheets, **relational tables**
 - Assumes a strict **schema** and relatively **dense** data (few missing/Null values)
- Flat file with **triplets** (record id, attribute, attribute value)
 - A very flexible data format, allows multiple values for the same attribute (e.g., phone number)
- **JSON, XML format**
 - Standards for data description that are more flexible than relational tables
 - There exist parsers for reading such data.

Examples

Comma Separated File

```
id,Name,Surname,Age,Zip  
1,John,Smith,25,10021  
2,Mary,Jones,50,96107  
3,Joe ,Doe,80,80235
```

- Can be processed with simple parsers, or loaded to excel or a database

Triple-store

```
1, Name, John  
1, Surname, Smith  
1, Age, 25  
1, Zip, 10021  
2, Name, Mary  
2, Surname, Jones  
2, Age, 50  
2, Zip, 96107  
3, Name, Joe  
3, Surname, Doe  
3, Age, 80  
3, Zip, 80235
```

- Easy to deal with missing values

Examples

JSON EXAMPLE – Record of a person

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```

XML EXAMPLE – Record of a person

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd
Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
    <phoneNumber>
      <type>fax</type>
      <number>646 555-4567</number>
    </phoneNumber>
  </phoneNumbers>
  <gender>
    <type>male</type>
  </gender>
</person>
```

JSON Databases

- There are specialized Databases for semi-structured data such as JSON data
- Commonly used DBs:
 - MongoDB
 - CouchDB
 - MySQL
- The query language is very similar to SQL used for relational data.

Beyond relational data: Set data

- Each record is a **set of items** from a space of possible items
- Example: Transaction data
 - Also called **market-basket data**

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Set data

- Each record is a **set of items** from a space of possible items
- Example: Document data
 - Also called **bag-of-words** representation

Doc Id	Words
1	the, dog, followed, the, cat
2	the, cat, chased, the, cat
3	the, man, walked, the, dog

Vector representation of market-basket data

- Market-basket data can be **represented**, or **thought of**, as **numeric vector data**
 - The vector is defined over the set of **all possible items**
 - The values are **binary** (the item appears or not in the set)

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

TID	Bread	Coke	Milk	Beer	Diaper
1	1	1	1	0	0
2	1	0	0	1	0
3	0	1	1	1	1
4	1	0	1	1	1
5	0	1	1	0	1

Sparsity: Most entries are zero. Most baskets contain few items

Vector representation of document data

- Document data can be **represented**, or **thought of**, as **numeric vector data**
 - The vector is defined over the set of **all possible words**
 - The values are the **counts** (number of times a word appears in the document)

Doc Id	Words
1	the, dog, follows, the, cat
2	the, cat, chases, the, cat
3	the, man, walks, the, dog

Doc Id	the	dog	follows	cat	chases	man	walks
1	2	1	1	1	0	0	0
2	2	0	0	2	1	0	0
3	1	1	0	0	0	1	1

Sparsity: Most entries are zero. Most documents contain few of the words

Physical data storage

- Usually set data is stored in flat files
 - One line per set

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
30 31 32
33 34 35
36 37 38 39 40 41 42 43 44 45 46
38 39 47 48
38 39 48 49 50 51 52 53 54 55 56 57 58
32 41 59 60 61 62
3 39 48
```

- I heard so many good things about this place so I was pretty juiced to try it. I'm from Cali and I heard Shake Shack is comparable to IN-N-OUT and I gotta say, Shake Shake wins hands down. Surprisingly, the line was short and we waited about 10 MIN. to order. I ordered a regular cheeseburger, fries and a black/white shake. So yummerz. I love the location too! It's in the middle of the city and the view is breathtaking. Definitely one of my favorite places to eat in NYC.
- I'm from California and I must say, Shake Shack is better than IN-N-OUT, all day, err'day.

Dependent data

- In tables we usually consider each object independent of each other.
- In some cases, there are explicit **dependencies** between the data
 - **Ordered/Temporal data**: We know the time order of the data
 - **Spatial data**: Data that is placed on specific locations
 - **Spatiotemporal data**: data with location and time
 - **Networked/Graph data**: data with pairwise relationships between entities

Ordered Data

- Genomic **sequence** data

```
GGTTCCGCCTTCAGCCCCGCGCC
CGCAGGGCCCGCCCCGCGCCGTC
GAGAAGGGCCCGCCTGGCGGGCG
GGGGGAGGCGGGGCCGCCCGAGC
CCAACCGAGTCCGACCAGGTGCC
CCCTCTGCTCGGCCTAGACCTGA
GCTCATTAGGCGGCAGCGGACAG
GCCAAGTAGAACACGCGAAGCGC
TGGGCTGCCTGCTGCGACCAGGG
```

- Data is a long **ordered** string

Ordered Data

- Time series
 - Sequence of ordered (over “time”) numeric values.



Ordered Data

- Sequence data: Similar to the time series but in this case, we have categorical values rather than numerical ones.
- Example: Event logs

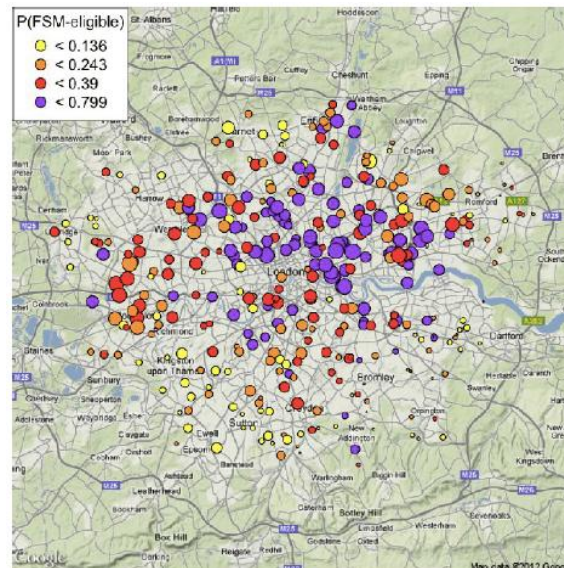
```
fcrawler.looksmart.com - - [26/Apr/2000:00:00:12 -0400] "GET /contacts.html HTTP/1.0" 200 4595 "-" "FAST-WebCraw
fcrawler.looksmart.com - - [26/Apr/2000:00:17:19 -0400] "GET /news/news.html HTTP/1.0" 200 16716 "-" "FAST-WebCra

ppp931.on.bellglobal.com - - [26/Apr/2000:00:16:12 -0400] "GET /download/windows/asctab31.zip HTTP/1.0" 200 1540

123.123.123.123 - - [26/Apr/2000:00:23:48 -0400] "GET /pics/wpaper.gif HTTP/1.0" 200 6248 "http://www.jafsoft.com
123.123.123.123 - - [26/Apr/2000:00:23:47 -0400] "GET /asctortf/ HTTP/1.0" 200 8130 "http://search.netscape.com/
123.123.123.123 - - [26/Apr/2000:00:23:48 -0400] "GET /pics/5star2000.gif HTTP/1.0" 200 4005 "http://www.jafsoft
123.123.123.123 - - [26/Apr/2000:00:23:50 -0400] "GET /pics/5star.gif HTTP/1.0" 200 1031 "http://www.jafsoft.com
123.123.123.123 - - [26/Apr/2000:00:23:51 -0400] "GET /pics/a2hlogo.jpg HTTP/1.0" 200 4282 "http://www.jafsoft.com
123.123.123.123 - - [26/Apr/2000:00:23:51 -0400] "GET /cgi-bin/newcount?jafsof3&width=4&font=digital&noshow HTTP/
```


Spatial data

- Attribute values that can be arranged with **geographic co-ordinates**
 - Measurements of temperature/pressure in different locations.
 - Sales numbers in different stores
 - The majority party in the country states (categorical)
- Such data can be nicely **visualized**.



Spatiotemporal data

- Data that have both spatial and temporal aspects
 - Measurements in different locations over time
 - Pressure, Temperature, Humidity
 - Measurements that move in space over time
 - Traffic, Trajectories of moving objects

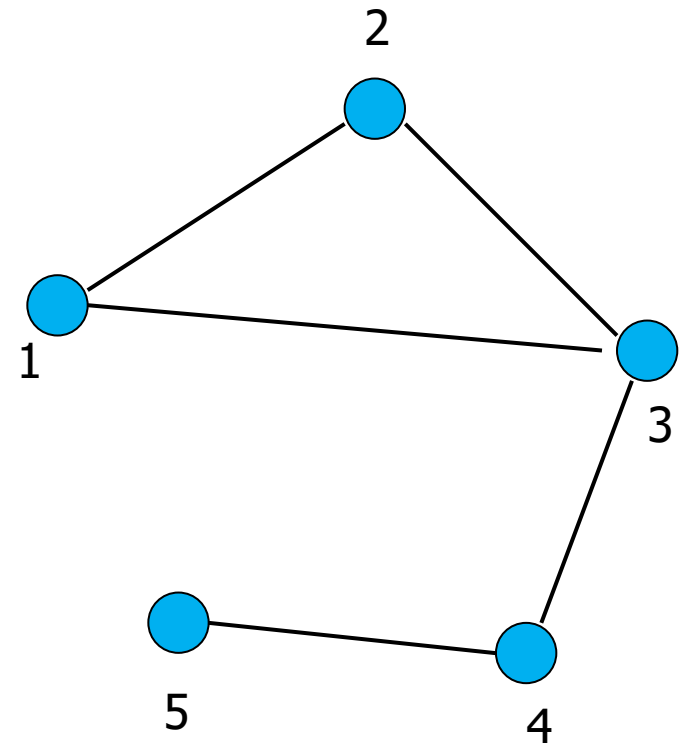
Graph Data

- Graph data: a collection of **entities** and their **pairwise relationships**.
- Examples:
 - Web pages and hyperlinks
 - Facebook users and friendships
 - The connections between brain neurons
 - Genes that regulate each other

In this case the data consists of **pairs**:

Who links to whom

We may have **undirected** links



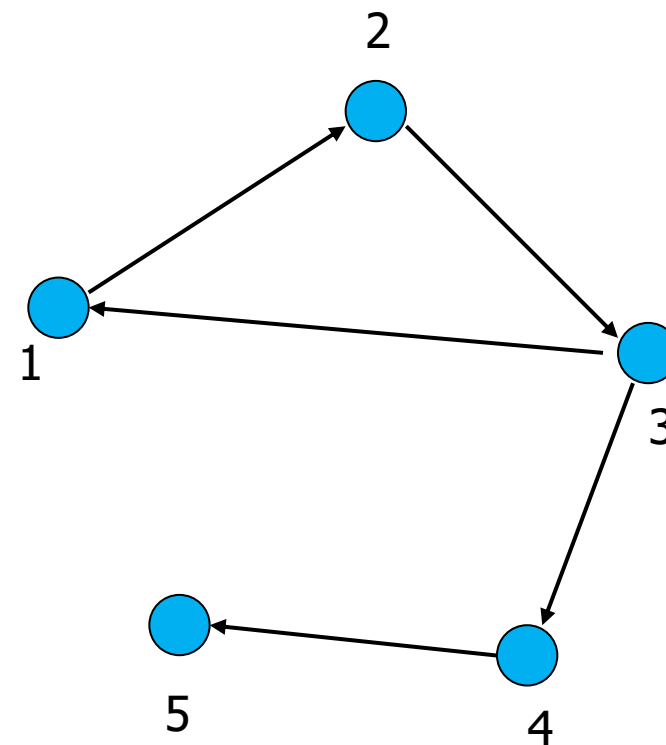
Graph Data

- Graph data: a collection of **entities** and their **pairwise relationships**.
- Examples:
 - Web pages and hyperlinks
 - Facebook users and friendships
 - The connections between brain neurons
 - Genes that regulate each other

In this case the data consists of **pairs**:

Who links to whom

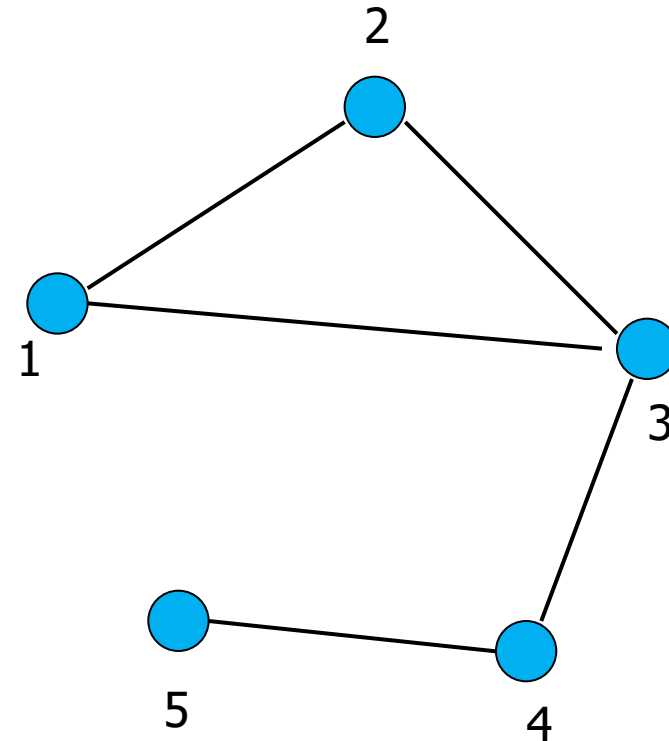
Or **directed** links



Representation

- Adjacency matrix
 - Very sparse, very wasteful, but useful conceptually

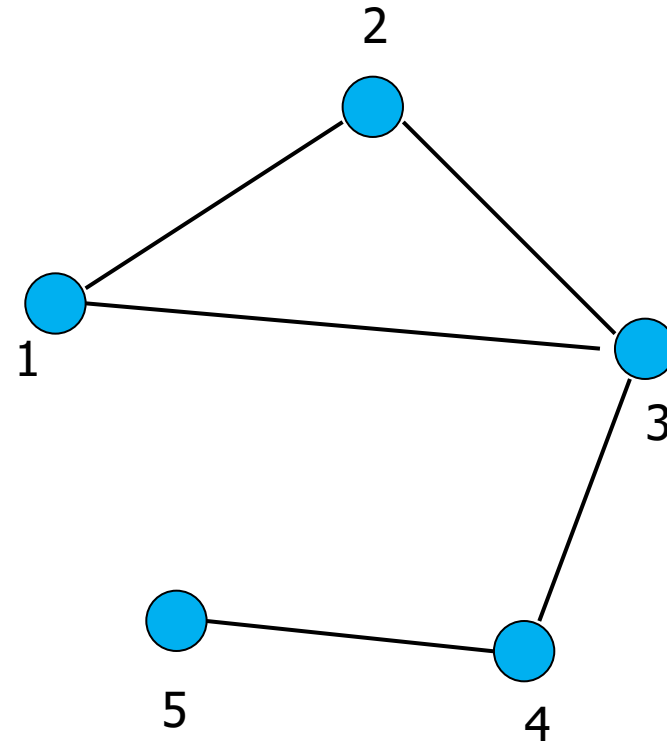
$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Representation

- Adjacency list
 - Not so easy to maintain

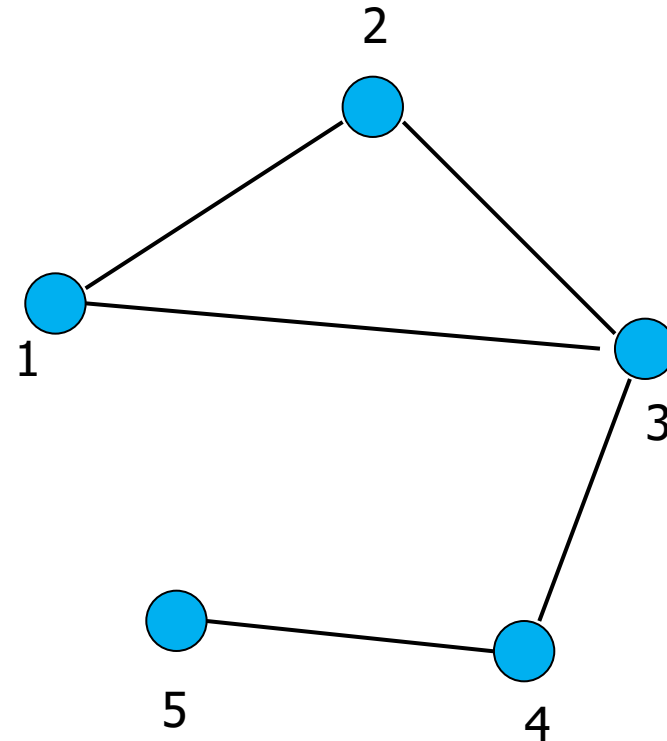
1: [2, 3]
2: [1, 3]
3: [1, 2, 4]
4: [3, 5]
5: [4]



Representation

- List of pairs
 - The simplest and most efficient representation

(1,2)
(2,3)
(1,3)
(3,4)
(4,5)



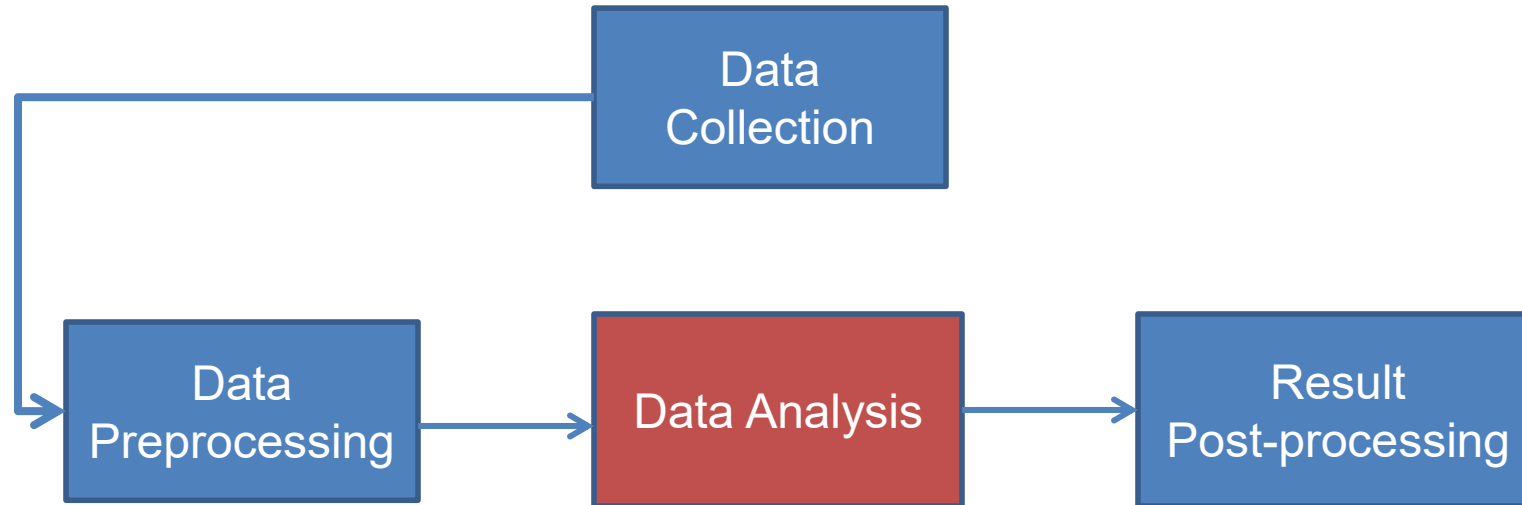
Types of data: summary

- **Numeric data:** Each object is a point in a multidimensional space
- **Categorical data:** Each object is a vector of categorical values
- **Set data:** Each object is a set of values (with or without counts)
 - Sets can also be represented as binary vectors, or vectors of counts
- **Dependent data:**
 - **Ordered sequences:** Each object is an ordered sequence of values.
 - **Spatial data:** objects are fixed on specific geographic locations
 - **Graph data:** A collection of pairwise relationships
- **The data matrix:**
 - In almost all types of data we can find a way to transform the data into a **matrix**, where the rows correspond to different records, and the columns to numeric attributes

DATA MINING PIPELINE

The data mining pipeline

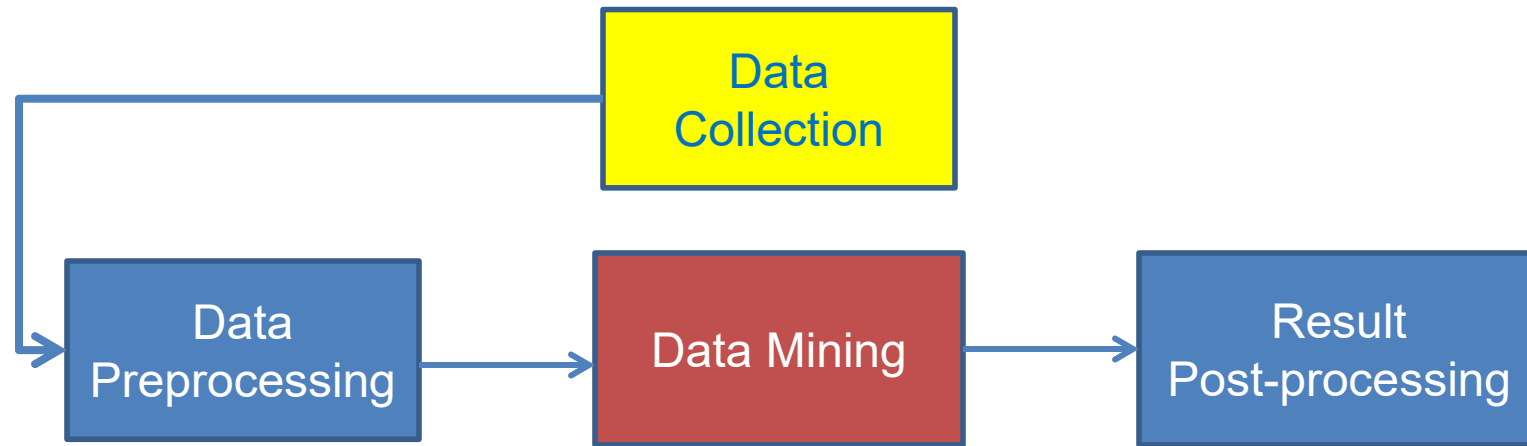
When talking about Data Mining we usually think of the Data Analysis part, but the Data Mining pipeline has several steps



In this course we will focus mostly on Data Analysis: methods and algorithms for extracting useful knowledge from the data.

DATA COLLECTION

The data mining pipeline



- Collecting the data is the start of the Data Mining pipeline
- It can be a hard problem; we will not focus on it in this course.

The different ways of obtaining data

- **Generate your own data**

- Data may be generated through logging of a process, or some other activity (e.g., scientific measurements)
- Important to collect the right amount of information.

- **Use existing collections**

- Today there are a lot of data collections available online
- Open data, corporate data releases, Wikis, scientific data sharing
- Existing data collections are subject to some choices made by the creator of the collection

- **Obtaining data online**

- Public APIs: A lot of online platforms (e.g., Twitter) provide APIs for accessing their data (under several restrictions). You obtain structured data fast
- Crawling & Scraping: Use a program that traverses web pages and downloads them (crawling), and then extracts the useful content from them (scraping)
 - You should follow the politeness etiquette when crawling.
 - Messy and not so robust, as page layouts change often

Data labels

- For many supervised learning tasks (classification), you need **labeled data**, which will be used for training and testing
- Examples:
 - For a collection of tweets, label them as offensive or not
 - For a collection of sentences label them as having a positive, negative, or neutral sentiment towards a specific item
 - For a collection of search results, label them as relevant or not relevant to the query
- These labels constitute the “**ground truth**” that we are trying to predict.
- Obtaining such labels is a difficult task that usually requires manual work.
 - Companies employ workers for this task

Data collection example

- Suppose that you want to collect data from **Twitter** about the elections in USA
 - How do you go about it?
- Twitter Streaming/Search API:
 - Get a sample of all tweets that are posted on Twitter
 - [Example](#) of JSON object
- REST API:
 - Get information about specific users.
- There are several decisions that we need to make before we start collecting the data.
 - Time and Storage resources

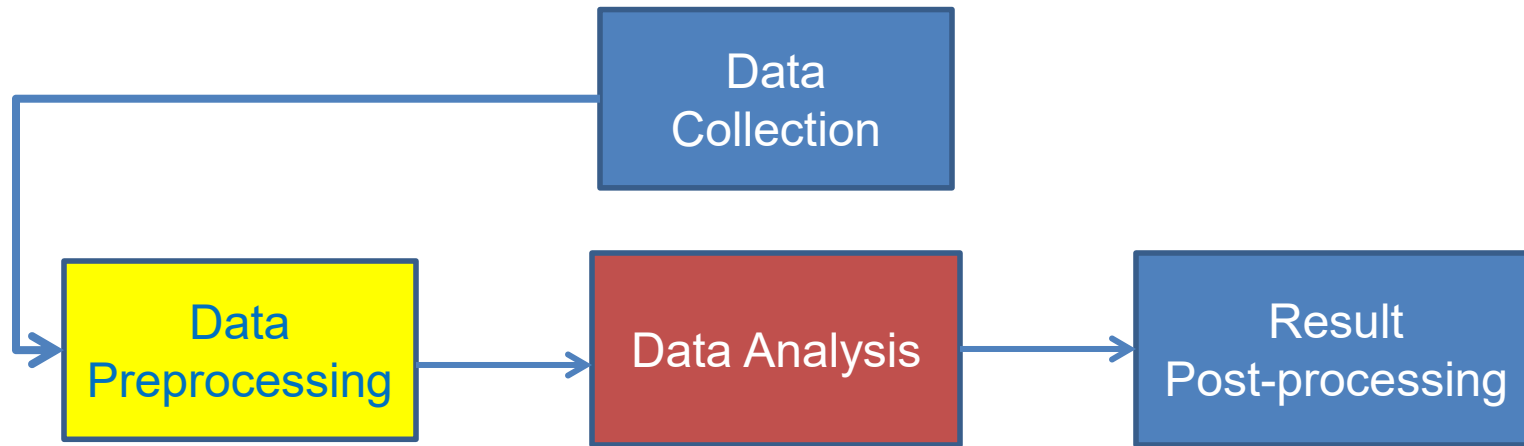
PREPROCESSING

Sampling – reservoir sampling

Feature extraction – TF-IDF

Data Normalization

The data mining pipeline



- **Preprocessing**: Real data is large, noisy, incomplete and inconsistent. We need to preprocess it before we use it
- The preprocessing step determines the **input** to the data mining algorithm
 - It is often the most important step for the analysis
 - A dirty work, but someone has to do it.

Preprocessing steps

- Reducing the data: Sampling, Dimensionality Reduction
- Data cleaning: deal with missing or inconsistent information
- Feature extraction and selection: create a useful representation of the data by extracting useful features

SAMPLING – DIMENSIONALITY REDUCTION

Sampling

- **Sampling** is the main technique employed for data selection.
 - It is often used for both the preliminary investigation of the data and the final data analysis.
- Statisticians sample because **obtaining** the entire set of data of interest is too expensive or time consuming.
 - Example: What is the average height of a person in Greece?
 - We cannot measure the height of everybody
- Sampling is used in data mining because **processing** the entire set of data of interest is too expensive or time consuming.
 - Example: We have **1M** documents. What fraction of pairs has at least 100 words in common?
 - Computing number of common words for all pairs requires **10^{12}** comparisons
 - Example: What fraction of tweets in a year contain the word “Greece”?
 - **500M** tweets per day, if **100** characters on average, **86.5TB** to store all tweets

Sampling ...

- The key principle for effective sampling is the following:
 - using a sample will work almost as well as using the entire data sets, if the sample is **representative**
 - A sample is representative if it has approximately the same property (of interest) as the original set of data
 - Otherwise, we say that the sample introduces some **bias**
 - What happens if we take a sample from the university campus to compute the average height of a person in Ioannina?

Types of Sampling

- Simple Random Sampling
 - There is an equal probability of selecting any particular item
- Sampling **without replacement**
 - As each item is selected, it is removed from the population
- Sampling **with replacement**
 - Objects are not removed from the population as they are selected for the sample.
 - In sampling with replacement, the same object can be picked up more than once. This makes analytical computation of probabilities easier
 - E.g., we have 100 people, 51 are women $P(W) = 0.51$, 49 men $P(M) = 0.49$. If I pick two persons what is the probability $P(W,W)$ that both are women?
 - Sampling with replacement: $P(W,W) = 0.51^2$
 - Sampling without replacement: $P(W,W) = 51/100 * 50/99$

Types of Sampling

- **Stratified** sampling

- Split the data into several **groups**; then draw random samples from each group.
 - Ensures that all groups are **represented**.
- **Example 1.** I want to understand the differences between legitimate and fraudulent credit card transactions. **0.1%** of transactions are fraudulent. What happens if I select **1000** transactions at random?
 - I get **1** fraudulent transaction (in expectation). Not enough to draw any conclusions. Solution: sample **1000** legitimate and **1000** fraudulent transactions

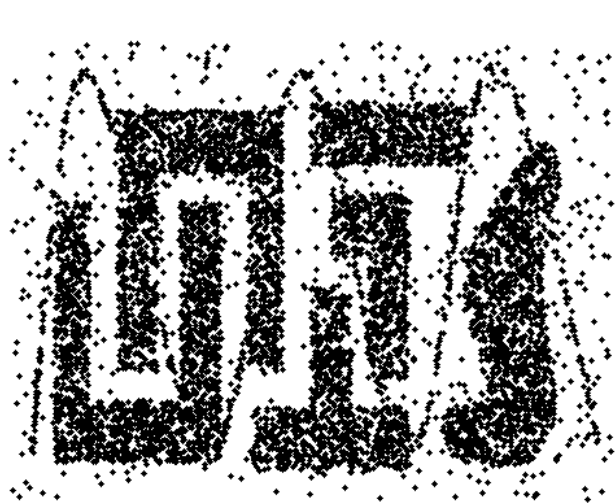
Probability Reminder: If an event has probability **p** of happening and I do **N** trials, the expected number of times the event occurs is **pN**

- **Example 2.** I want to answer the question: Do web pages that are linked have on average more words in common than those that are not? I have **1M** pages, and **5M** links, what happens if I select **10K pairs of pages** at random?
 - Most likely I will not get any links.
 - Solution: sample **10K** random pairs, and **10K** links

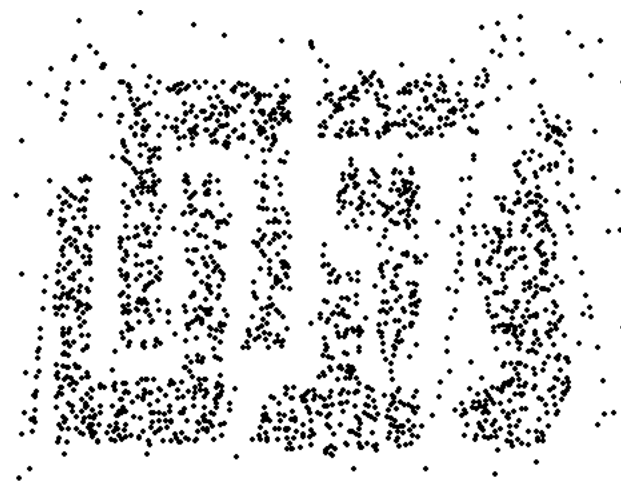
Biased sampling

- Sometimes we want to bias our sample towards some subset of the data
 - Stratified sampling is one example
- Example: When sampling temporal data, we want to increase the probability of sampling recent data
 - Introduce **recency bias**
- Make the sampling probability to be a function of time, or the age of an item
 - Typical: Probability **decreases exponentially with time**
 - For item x_t after time t select with probability $p(x_t) \propto e^{-t}$

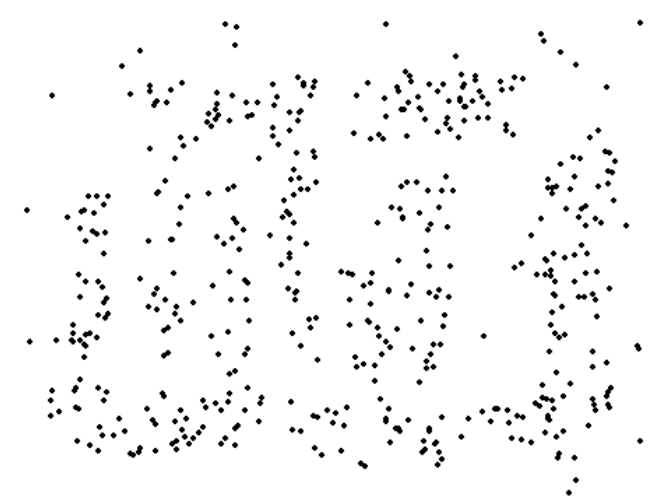
Sample Size



8000 points



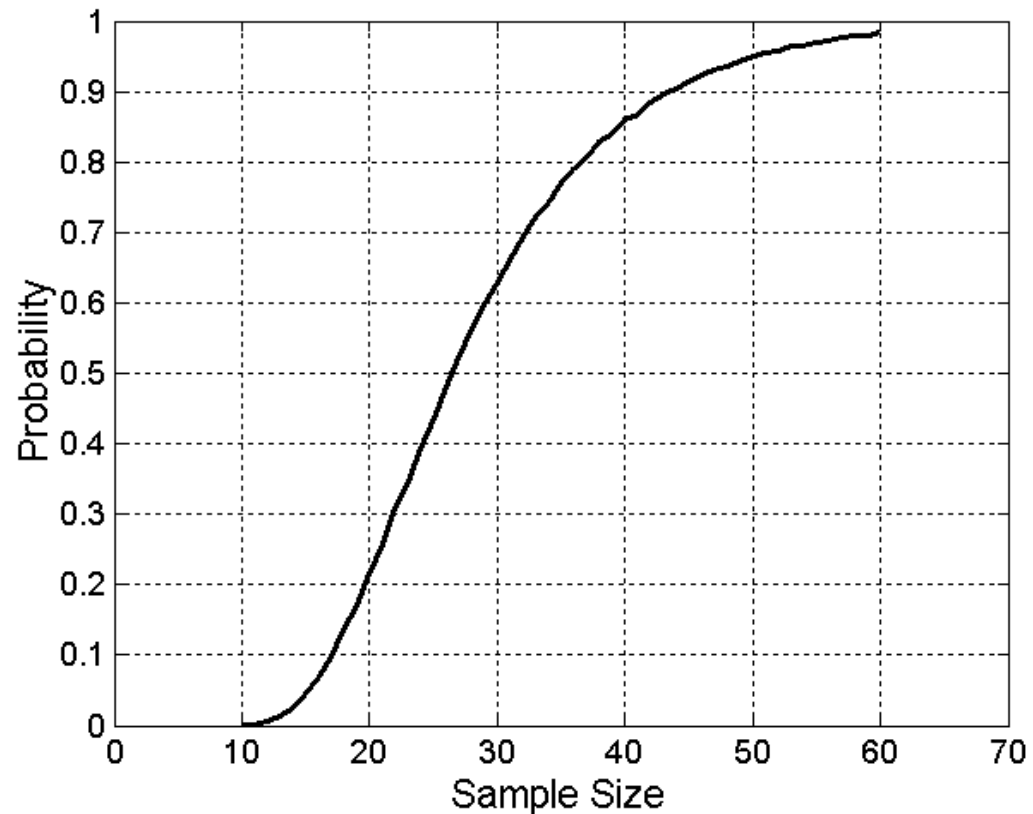
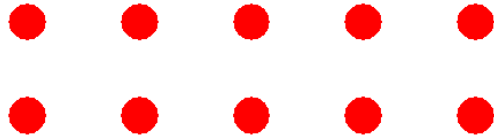
2000 Points



500 Points

Sample Size

- What sample size is necessary to get at least one object from each of 10 groups.



A data mining challenge

- You have N items and you want to sample one item uniformly at random. How do you do that?
- The items are coming in a **stream**: you do not know the size of the stream in advance, and there is not enough memory to store the stream in memory. You can only keep a **constant** amount of items in memory
- How do you sample?
 - Hint: if the stream ends after reading k items the last item in the stream should have probability $1/k$ to be selected.
- **Reservoir Sampling**:
 - (Used to be) a standard interview question for many companies

Reservoir sampling

- **Algorithm** (in plain English):
 - Obtain the items from the stream one at the time.
 - Select the 1st item and store it in the item variable
 - When seeing the **k-th** item select it with probability **1/k** and replace the existing selection.
- The algorithm stores only one item, and one integer number (the count of items seen so far)

Reservoir sampling proof

- **Claim:** Every item has probability $1/N$ to be selected after N items have been read.

- **Proof**

- What is the probability of the k -th item to be selected when seen for the first time?

$$P(k \text{ item selected when first seen}) = \frac{1}{k} \quad (P(k \text{ selected}))$$

- What is the probability of the selected item to survive round m :

$$P(\text{selected item survives round } m) = \left(1 - \frac{1}{m}\right) = \frac{m-1}{m} \quad (P(\text{survive } m))$$

- The probability that the k -th item is selected after N items are seen

$$P(k \text{ item selected after } N \text{ rounds})$$

$$= P(k \text{ selected})P(k \text{ survives } \textbf{until} \text{ round } N)$$

$$= P(k \text{ selected})P(k \text{ survives } k+1) P(k \text{ survives } k+2) \cdots P(k \text{ survives } N)$$

$$= \frac{1}{k} \frac{k}{k+1} \frac{k+1}{k+2} \cdots \frac{N-1}{N}$$

$$= \frac{1}{N}$$

The proof holds for any $k, 1 \leq k \leq N$

Proof by Induction

- We want to show that the probability the k -th item is the selected item after $n \geq k$ items have been seen is $\frac{1}{n}$
- Induction on the number of rounds n
 - Base of the induction: For $n = k$, the probability that the k -th item is the selected item is the probability that it is selected when first seen: $\frac{1}{k}$
 - Inductive Hypothesis: Assume that the hypothesis is true for $n = m, m \geq k$:
 - The probability that the k -th item is the selected item at round m is $\frac{1}{m}$
 - Inductive Step: The probability that the k -th item is the selected item at round $n = m + 1$ is

$$P(k \text{ selected at round } m)P(k \text{ survives } m) = \frac{1}{m} \left(1 - \frac{1}{m+1} \right) = \frac{1}{m+1}$$

The proof holds for any $k, 1 \leq k \leq N$

Dimensionality Reduction

- Sampling reduces the number of records. We can also reduce the dimension of the data, the number of attributes
- Real data is **high-dimensional**: typically, it has several hundreds, thousands, or even million of attributes.
 - Documents represented as vectors of word counts (millions)
 - Facebook users represented as vectors of friends (billions)
 - Customers represented as vectors of products (hundreds of thousands)
- Data is extremely sparse and noisy
- Dimensionality reduction aims to:
 - Reduce the amount of data
 - Extract the useful information.

Example

- Consider the following 6-dimensional dataset

$$D = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 & 0 \\ 2 & 4 & 6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 2 & 4 & 6 \\ 1 & 2 & 3 & 1 & 2 & 3 \\ 2 & 4 & 6 & 2 & 4 & 6 \end{bmatrix}$$

- What do you **observe**? Can we reduce the dimension of the data?

Example

$$D = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 & 0 \\ 2 & 4 & 6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 2 & 4 & 6 \\ 1 & 2 & 3 & 1 & 2 & 3 \\ 2 & 4 & 6 & 2 & 4 & 6 \end{bmatrix}$$

- Each row is a **multiple** of two **vectors**
 - $x = [1, 2, 3, 0, 0, 0]$
 - $y = [0, 0, 0, 1, 2, 3]$
- We can rewrite D as

$$D = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 0 & 1 \\ 0 & 2 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \end{bmatrix}$$

DATA CLEANING

Data Quality

- Examples of data quality problems:
 - Noise and outliers
 - Missing values
 - Duplicate data

A mistake or a millionaire?

Missing values

Inconsistent duplicate entries

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	10000K	Yes
6	No	NULL	60K	No
7	Yes	Divorced	220K	NULL
8	No	Single	85K	Yes
9	No	Married	90K	No
9	No	Single	90K	No

Data cleaning

- How do we deal with data that are noisy?
- The benefit of having a lot of data is that we can **throw out data** that we suspect are problematic or that we do not consider to be useful
- For example:
 - Throw out all records with missing values, or duplicate values.
 - Throw out **extreme**, or **atypical** cases
 - In database data, throw out records with outlier values
 - In tweet text data throw out tweets with less than 3 words
 - In transaction data, throw out baskets with very few items, and throw out products (attributes) that are bought by everyone and products that are bought by very few
 - In social network data, keep only users with enough friends but not too many
- We should always be careful not to throw out useful information.

Data Cleaning

- Deal with compatibility issues
 - **Units** may be different in different parts of the data (centimeters vs meters, or meters vs feet)
 - **Time** measurements should be brought into the same time zone.
 - Normalize **names**:
 - Panayiotis Tsaparas, P. Tsaparas, and P. N. Tsaparas are all the same person
 - **Financial units** should be normalized
 - Different currencies
 - Prices over time

Data Cleaning

- Deal with **missing values**:
 - Ignore the data
 - Replace with random value
 - Not a good idea, but you can understand how the missing value affects the output
 - Replace with the mean
 - Relatively common practice.
 - Should be careful of cases where this does not make sense (e.g., year of birth/death)
 - Replace with nearest neighbor value
 - Replace with cluster mean
 - Infer the value

Data Cleaning

- Deal with missing values: Examples

2	1	8	3
-2	0	-4	-3
2.5	1	7	?

What value would you fill?

Use **nearest neighbor** and fill in 3

2	1	8	3
3	1.5	6	2
-2	0	-4	-3
-1	1	-3	-1
2.5	1	7	?

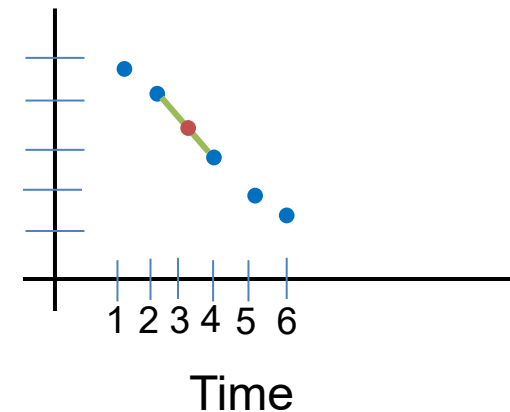
What value would you fill?

Use **cluster mean** and fill in 2.5

2	1	8	4
3	1.5	6	6
4	2	5	8
2	1	4	4
2.5	1	7	?

What value would you fill?

The last column is twice the first one: fill in 5



What value would you fill for timestamp 3?

Linear interpolation

Data Cleaning

- Deal with outliers:
 - Remove them
 - Try to correct them using common sense
 - Transform the data
 - In some data (e.g., wealth, social media followers) extreme values are expected and interesting
 - In such skewed distributions we typically use the **logarithm** of the values, or apply **binning**

Data Cleaning

- When using the data, we should be careful of cases where our results are **too good to be true**, or **too bad to be true**

Data Cleaning

- When using the data, we should be careful of cases where our results are **too good to be true**, or too bad to be true

<i>Tid</i>	Refund	Marital Status	Taxable Income	Fined
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	100K	Yes
6	No	NULL	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	90K	No
10	No	Single	90K	No

<i>Tid</i>	Cheat
1	No
2	No
3	No
4	No
5	Yes
6	No
7	No
8	Yes
9	No
10	No

We trained a classifier to predict if a taxpayer will cheat and we got perfect results. Is our classifier great or is there some problem with the data?

The attribute Fined is the same as the class label we want to predict

Data Cleaning

- When using the data, we should be careful of cases where our results are too good to be true, or **too bad to be true**

<i>Tid</i>	Refund	Marital Status	Taxable Income
1	Yes	Single	125K
2	No	Married	100K
3	No	Single	70K
4	Yes	Married	120K
5	No	Divorced	100K
6	No	NULL	60K
7	Yes	Divorced	220K
8	No	Single	85K
9	No	Married	90K
10	No	Single	90K

<i>Tid</i>	Cheat
0	No
1	No
2	No
3	No
4	Yes
5	No
6	No
7	Yes
8	No
9	No

We trained a classifier to predict if a taxpayer will cheat and we got bad results. Is our classifier that bad?

The ids for the class labels are not aligned with the ids of the data

FEATURE EXTRACTION

A case study with text data

TF-IDF word weighting

Data preprocessing: feature extraction

- The data we obtain are not necessarily as a relational table
- Data may be in a very raw format
 - Examples: text, speech, mouse movements, etc
- We need to extract the **features/attributes** from the data and build the data matrix
- Feature extraction:
 - Selecting the characteristics by which we want to represent our data
 - It requires some domain knowledge about the data
 - It depends on the application
- Deep Learning Embeddings help with this step.

Example: Text data

- **Text** data is a collection of strings
 - We need to do additional effort to extract the useful information
- We will now see some basic **text processing** ideas.
- Example: Mine the restaurant reviews of users on [Yelp](https://www.yelp.com)



Mining Task

- Collect all reviews for the top-10 most reviewed restaurants in NY in Yelp

```
{ "votes": { "funny": 0, "useful": 2, "cool": 1 },  
  "user_id": "Xqd0DzHaiyRqVH3WRG7hzhg",  
  "review_id": "15SdjuK7DmYqUAj6rjGowg",  
  "stars": 5, "date": "2007-05-17",  
  "text": "I heard so many good things about this place so I was pretty juiced to try  
it. I'm from Cali and I heard Shake Shack is comparable to IN-N-OUT and I gotta  
say, Shake Shake wins hands down. Surprisingly, the line was short and we waited  
about 10 MIN. to order. I ordered a regular cheeseburger, fries and a black/white  
shake. So yummerz. I love the location too! It's in the middle of the city and  
the view is breathtaking. Definitely one of my favorite places to eat in NYC.",  
  "type": "review",  
  "business_id": "vcNAWiLM4dR7D2nwwJ7nCA" }
```

- **Feature extraction:** Find few words that best describe the restaurants.

Example data

I heard so many good things about this place so I was pretty juiced to try it. I'm from Cali and I heard Shake Shack is comparable to IN-N-OUT and I gotta say, Shake Shack wins hands down. Surprisingly, the line was short and we waited about 10 MIN. to order. I ordered a regular cheeseburger, fries and a black/white shake. So yummerz. I love the location too! It's in the middle of the city and the view is breathtaking. Definitely one of my favorite places to eat in NYC.

I'm from California and I must say, Shake Shack is better than IN-N-OUT, all day, err'day.

Would I pay \$15+ for a burger here? No. But for the price point they are asking for, this is a definite bang for your buck (though for some, the opportunity cost of waiting in line might outweigh the cost savings) Thankfully, I came in before the lunch swarm descended and I ordered a shake shack (the special burger with the patty + fried cheese & portabella topping) and a coffee milk shake. The beef patty was very juicy and snugly packed within a soft potato roll. On the downside, I could do without the fried portabella-thingy, as the crispy taste conflicted with the juicy, tender burger. How does shake shack compare with in-and-out or 5-guys? I say a very close tie, and I think it comes down to personal affiliations. On the shake side, true to its name, the shake was well churned and very thick and luscious. The coffee flavor added a tangy taste and complemented the vanilla shake well. Situated in an open space in NYC, the open air sitting allows you to munch on your burger while watching people zoom by around the city. It's an oddly calming experience, or perhaps it was the food

Decisions, decisions...

- We need to make some **decisions** about our data collection
 - **What** data should we collect? **How much**? For **how long**?
 - What data should we **keep**?

An actual review

```
AAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

- How should we **weight** the different pieces of data?
- We should make our decisions **clear** upfront. Most decisions are task-dependent. Some information may be **lost** but we can usually live with it (most of the times)

Text normalization

- Each review is a long string. We need to transform it into words
- Basic preprocessing:
 - “normalize” the data (remove punctuation, make into lower case, clear white spaces, other?)
 - Break into words
- There are existing libraries that do these steps.
- Sometimes, we can break into **n-grams**, or combinations of words.

First cut

- Keep the most popular words

the 27514
and 14508
i 13088
a 12152
to 10672
of 8702
ramen 8518
was 8274
is 6835
it 6802
in 6402
for 6145
but 5254
that 4540
you 4366
with 4181
pork 4115
my 3841
this 3487
wait 3184
not 3016
we 2984
at 2980
on 2922

the 16710
and 9139
a 8583
i 8415
to 7003
in 5363
it 4606
of 4365
is 4340
burger 432
was 4070
for 3441
but 3284
shack 3278
shake 3172
that 3005
you 2985
my 2514
line 2389
this 2242
fries 2240
on 2204
are 2142
with 2095

the 16010
and 9504
i 7966
to 6524
a 6370
it 5169
of 5159
is 4519
sauce 4020
in 3951
this 3519
was 3453
for 3327
you 3220
that 2769
but 2590
food 2497

cart 2236
chicken 2220
with 2195
rice 2049
so 1825

the 14241
and 8237
a 8182
i 7001
to 6727
of 4874
you 4515
it 4308
is 4016
was 3791
pastrami 3748
in 3508
for 3424
sandwich 2928
that 2728
but 2715
on 2247

with 2040
not 1655
your 1622
so 1610
have 1585

Most frequent words are **stop words**

Second cut

- Remove stop words
 - Stop-word lists can be found online.

a, about, above, after, again, against, all, am, an, and, any, are, aren't, as, at, be, because, been, before, being, below, between, both, but, by, can't, cannot, could, couldn't, did, didn't, do, does, doesn't, doing, don't, down, during, each, few, for, from, further, had, hadn't, has, hasn't, have, haven't, having, he, he'd, he'll, he's, her, here, here's, hers, herself, him, himself, his, how, how's, i, i'd, i'll, i'm, i've, if, in, into, is, isn't, it, it's, its, itself, let's, me, more, most, mustn't, my, myself, no, nor, not, of, off, on, once, only, or, other, ought, our, ours, ourselves, out, over, own, same, shan't, she, she'd, she'll, she's, should, shouldn't, so, some, such, than, that, that's, the, their, theirs, them, themselves, then, there, there's, these, they, they'd, they'll, they're, they've, this, those, through, to, too, under, until, up, very, was, wasn't, we, we'd, we'll, we're, we've, were, weren't, what, what's, when, when's, where, where's, which, while, who, who's, whom, why, why's, with, won't, would, wouldn't, you, you'd, you'll, you're, you've, your, yours, yourself, yourselves,

Second cut

- Remove stop words
 - Stop-word lists can be found online.

ramen 8572
pork 4152
wait 3195
good 2867
place 2361
noodles 2279
ippudo 2261
buns 2251
broth 2041
like 1902
just 1896
get 1641
time 1613
one 1460
really 1437
go 1366
food 1296
bowl 1272
can 1256
great 1172
best 1167

burger 4340
shack 3291
shake 3221
line 2397
fries 2260
good 1920
burgers 1643
wait 1508
just 1412
cheese 1307
like 1204
food 1175
get 1162
place 1159
one 1118
long 1013
go 995
time 951
park 887
can 860
best 849

sauce 4023
food 2507
cart 2239
chicken 2238
rice 2052
hot 1835
white 1782
line 1755
good 1629
lamb 1422
halal 1343
just 1338
get 1332
one 1222
like 1096
place 1052
go 965
can 878
night 832
time 794
long 792
people 790

pastrami 3782
sandwich 2934
place 1480
good 1341
get 1251
katz's 1223
just 1214
like 1207
meat 1168
one 1071
deli 984
best 965
go 961
ticket 955
food 896
sandwiches 813
can 812
beef 768
order 720
pickles 699
time 662

Second cut

- Remove stop words
 - Stop-word lists can be found online.

ramen 8572	burger 4340	sauce 4023	pastrami 3782
pork 4152	shack 3291	food 2507	sandwich 2934
wait 3195	shake 3221	cart 2239	place 1480
good 2867	line 2397	chicken 2238	good 1341
place 2361	fries 2260	rice 2052	get 1251
noodles 2279	good 1920	hot 1835	katz's 1223
ippudo 2261	burgers 1643	white 1782	just 1214
buns 2251	wait 1508	line 1755	like 1207
broth 2041	just 1412	good 1629	meat 1168
like 1902	cheese 1307	lamb 1422	one 1071
just 1896	like 1204	halal 1343	deli 984
get 1641	food 1175	just 1338	best 965
time 1613	get 1162	get 1332	go 961
one 1460	place 1159	one 1222	ticket 955
really 1437	one 1118	like 1096	food 896
go 1366	long 1012	place 1052	time 812
food 1296			
bowl 1272			
can 1256	park 887	night 832	order 720
great 1172	can 860	time 794	pickles 699
best 1167	best 849	long 792	time 662
		people 790	

Commonly used words in reviews, not so interesting

IDF: Inverse Document Frequency

- Important words for a document should be **unique** to the document (differentiating) compared to the rest of the **collection**
 - All reviews use the word “like”. This is not interesting
 - We want the words that characterize the specific restaurant

- **Document Frequency** $DF(w)$: fraction of documents that contain word w .

$$DF(w) = \frac{D(w)}{D}$$

$D(w)$: num of docs that contain word w

D : total number of documents

- **Inverse Document Frequency** $IDF(w)$:

$$IDF(w) = \log\left(\frac{1}{DF(w)}\right)$$

- Maximum when unique to one document : $IDF(w) = \log(D)$
- Minimum when the word is common to all documents: $IDF(w) = 0$

TF-IDF

- The words that are best for describing a document are the ones that are **important for the document**, but also **unique to the document**.
- $TF(w, d)$: **term frequency** of word w in document d
 - Natural measure of **importance** of the word for the document
- $IDF(w)$: inverse document frequency
 - Natural measure of the **uniqueness** of the word w
- $TF-IDF(w, d) = TF(w, d) \times IDF(w)$

Third cut

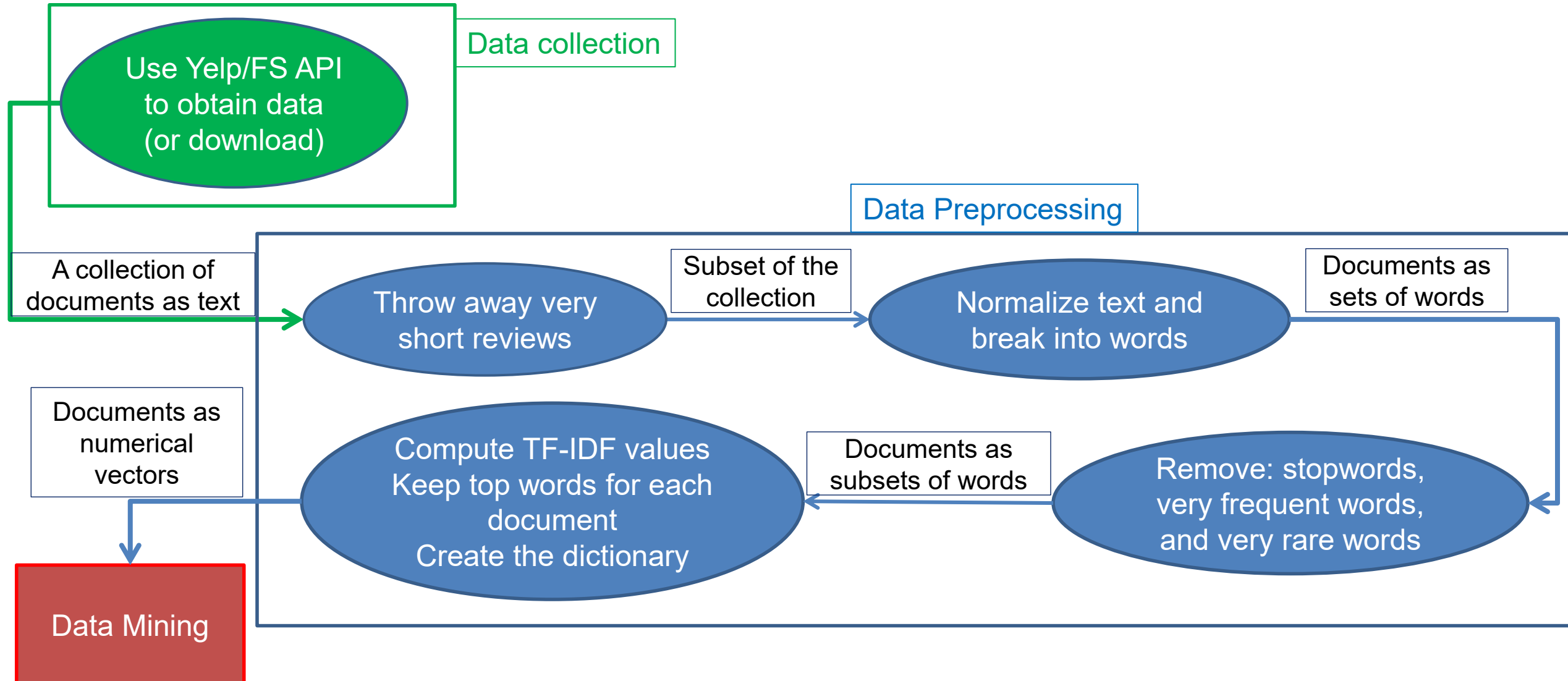
- Ordered by TF-IDF

ramen 3057.4176194	fries 806.08537330	lamb 985.655290756243	pastrami 1931.94250908298 6
akamaru 2353.24196	custard 729.607519	halal 686.038812717726	katz's 1120.62356508209 4
noodles 1579.68242	shakes 628.4738038	53rd 375.685771863491	rye 1004.28925735888 2
broth 1414.7133955	shroom 515.7790608	gyro 305.809092298788	corned 906.113544700399 2
miso 1252.60629058	burger 457.2646379	pita 304.984759446376	pickles 640.487221580035 4
hirata 709.1962086	crinkle 398.347221	cart 235.902194557873	reuben 515.779060830666 1
hakata 591.7643688	burgers 366.624854	platter 139.45990308004	matzo 430.583412389887 1
shiromaru 587.1591	madison 350.939350	chicken/lamb 135.852520	sally 428.110484707471 2
noodle 581.8446147	shackburger 292.42	carts 120.274374158359	harry 226.323810772916 4
tonkotsu 529.59457	'shroom 287.823136	hilton 84.2987473324223	mustard 216.079238853014 6
ippudo 504.5275695	portobello 239.806	lamb/chicken 82.8930633	cutter 209.535243462458 1
buns 502.296134008	custards 211.83782	yogurt 70.0078652365545	carnegie 198.655512713779 3
ippudo's 453.60926	concrete 195.16992	52nd 67.5963923222322	katz 194.387844446609 7
modern 394.8391629	bun 186.9621782983	6th 60.7930175345658 9	knish 184.206807439524 1
egg 367.3680056967	milkshakes 174.996	4am 55.4517744447956 5	sandwiches 181.415707218 8
shoyu 352.29551922	concretes 165.7861	yellow 54.4470265206673	brisket 131.945865389878 4
chashu 347.6903490	portabello 163.483	tzatziki 52.95945713886	fries 131.613054313392 7
karaka 336.1774235	shack's 159.334353	lettuce 51.323016802268	salami 127.621117258549 3
kakuni 276.3102111	patty 152.22603588	sammy's 50.656872045869	knishes 124.339595021678 1
ramens 262.4947006	ss 149.66803104461	sw 50.5668577816893 3	delicatessen 117.488967607 2
bun 236.5122638036	patties 148.068287	platters 49.90659700031	deli's 117.431839742696 1
wasabi 232.3667512	cam 105.9496067806	falafel 49.479699521204	carver 115.129254649702 1
dama 221.048168927	milkshake 103.9720	sober 49.2211422635451	brown's 109.441778045519 2
brulee 201.1797390	lamps 99.011158998	moma 48.1589121730374	matzoh 108.22149937072 1

Third cut

- TF-IDF takes care of stop words as well
- We do not need to remove the stopwords since they will get $IDF(w) \approx 0$
- **Important:** IDF is collection-dependent!
 - For some other corpus the words *get*, *like*, *eat*, may be important

The preprocessing pipeline for our text mining task



Word and document representations

- Using **TF-IDF** values has a very long history in text mining
 - Assigns a numerical value to each word, and a vector to a document
- Recent trend: Use **word embeddings**
 - Map every word into a multidimensional vector
- Use the notion of **context**: the words that surround a word in a phrase
 - Similar words appear in similar contexts
 - Similar words should be mapped to close-by vectors
- Example: words “movie” and “film”

The **actor** for the **movie** Joker is **candidate** for an **Oscar**
film
- Both words are likely to appear with similar words
 - director, actor, actress, scenario, script, Oscar, cinemas etc

word2vec

- Two approaches

CBOV: Learn an embedding for words so that given the context you can predict the missing word

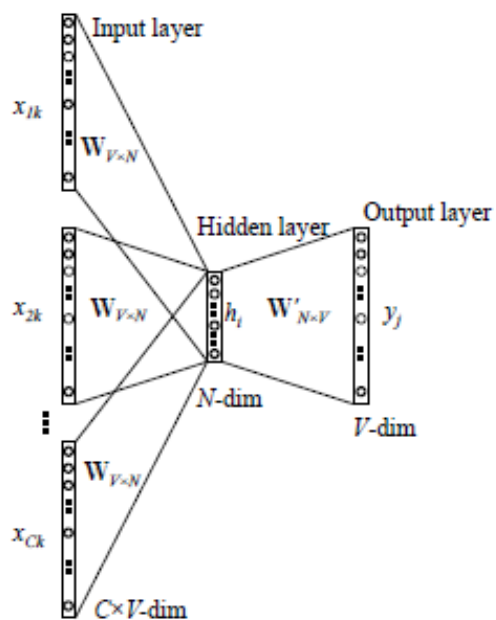


Figure 2: Continuous bag-of-words model

Skip-Gram: Learn an embedding for words such that given a word you can predict the context

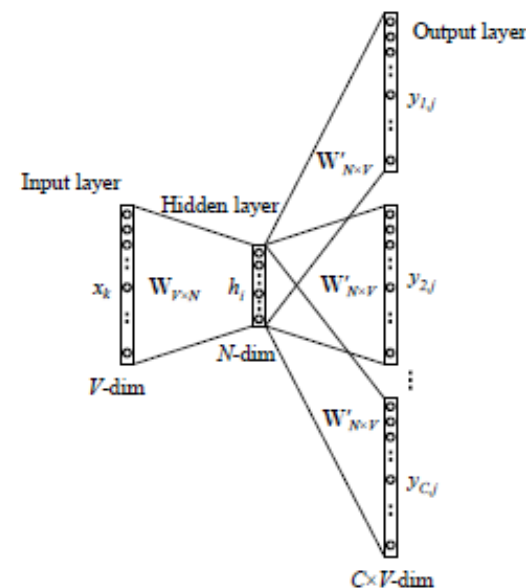


Figure 3: The skip-gram model.

Much more sophisticated methods have been developed since then, but they rely on the same principle

Embeddings- Representation Learning

- Word embeddings is one case of **representation learning**, which is becoming increasingly popular and tends to replace feature extraction
- General idea:
 - Take objects in some form (text, image, video, graph)
 - Define a (self) learning task
 - E.g., predict the next word, or a hidden word in text
 - E.g., predict the completion of an image, or the next frame in a video
 - Use an intermediate product of the learning algorithm (typically a layer in a NN) as the representation of the object
 - Represent objects with dense numerical vectors of high dimension.
 - The representations encode properties of the data, such as similarity
- There are also linear transformations of the data but non-linear ones have been shown to be much more powerful.

DATA NORMALIZATION

Normalization of numeric data

- In many cases it is important to **normalize** the data rather than use the raw values
- The kind of normalization that we use depends on what we want to achieve

Column normalization

- In this data, different attributes take very **different range of values**. When computing distance/similarity the small values will disappear
- We need to make them **comparable**

Temperature	Humidity	Pressure
30	0.8	90
32	0.5	80
24	0.3	95

Column Normalization

- Divide (the values of a **column**) by the **maximum value** for each attribute
 - Brings everything in the **[0,1] range, maximum is 1**

Temperature	Humidity	Pressure
0.9375	1	0.9473
1	0.625	0.8421
0.75	0.375	1

new value = old value / max value in the column

Temperature	Humidity	Pressure
30	0.8	90
32	0.5	80
24	0.3	95

Column Normalization

- Subtract the minimum value and divide by the difference of the maximum value and minimum value for each attribute
 - Brings everything in the $[0,1]$ range, maximum is one, minimum is zero

Temperature	Humidity	Pressure
0.75	1	0.33
1	0.6	0
0	0	1

new value = (old value – min column value) / (max col. value –min col. value)

Temperature	Humidity	Pressure
30	0.8	90
32	0.5	80
24	0.3	95

Column Normalization

- **Subtract** the **mean value** for each column – **centering** of features
 - All columns have **mean zero**

Temperature	Humidity	Pressure
1.33	0.27	1.67
3.33	-0.03	-8.33
-4.67	-0.23	6.67

	Temperature	Humidity	Pressure
	30	0.8	90
	32	0.5	80
	24	0.3	95
mean	28.67	0.53	88.33

new value = (old value – avg column value)

Column Normalization

- **Subtract** the **mean value** for each column – **centering** of features
 - All columns have **mean zero**
- **Divide** with the **length of the centered column vector**
 - All columns are **unit vectors**

Temperature	Humidity	Pressure
0.23	0.75	0.15
0.57	-0.09	-0.77
-0.79	-0.66	0.62

$$\text{new value} = \frac{\text{old value} - \text{mean value}}{\sqrt{\sum (\text{old value}_i - \text{mean value})^2}}$$

	Temperature	Humidity	Pressure
	30	0.8	90
	32	0.5	80
	24	0.3	95
mean	28.67	0.53	88.33

	Temperature	Humidity	Pressure
	1.33	0.27	1.67
	3.33	-0.03	-8.33
	-4.67	-0.23	6.67
length	5.89	0.36	10.80

Column Normalization

There is a relationship between z-score and centered unit vectors, what is it?

- **Subtract** the **mean value** for each column – **centering** of features
 - All columns have **mean zero**
- **Divide** with the **standard deviation of the column vector**
 - Computes the **z-score**
 - **Number of standard deviations away from the mean**

Temperature	Humidity	Pressure
0.23	0.75	0.15
0.57	-0.09	-0.77
-0.79	-0.66	0.62

$$\text{new value} = \frac{\text{old value} - \text{mean value}}{\text{standard deviation}}$$

$$\text{mean}(x) = \frac{1}{N} \sum_{j=1}^N x_j$$
$$\text{std}(x) = \sqrt{\frac{\sum_{j=1}^N (x_j - \text{mean}(x))^2}{N}}$$
$$\text{Z-score: } z_i = \frac{x_i - \text{mean}(x)}{\text{std}(x)}$$

	Temperature	Humidity	Pressure
	30	0.8	90
	32	0.5	80
	24	0.3	95
mean	28.67	0.53	88.33
std	3.40	0.21	6.24

Row Normalization

- Are these documents similar?

	Word 1	Word 2	Word 3
Doc 1	28	50	22
Doc 2	12	25	13

Row Normalization

- Are these documents similar?
- **Divide** by the **sum of values** for each document (row in the matrix)
 - Transform a vector into a **distribution***

	Word 1	Word 2	Word 3
Doc 1	0.28	0.5	0.22
Doc 2	0.24	0.5	0.26

new value = old value / Σ old values in the row

*For example, the value of cell (Doc1, Word2) is the **probability** that a **randomly chosen word** of Doc1 is Word2

	Word 1	Word 2	Word 3
Doc 1	28	50	22
Doc 2	12	25	13

Row Normalization

- Do these two users rate movies in a similar way?

	Movie 1	Movie 2	Movie 3
User 1	1	2	3
User 2	2	3	4

Row Normalization

- Do these two users rate movies in a similar way?
- **Subtract** the **mean value** for each user (row) – **centering** of the data rows
 - Captures the deviation from the average behavior

	Movie 1	Movie 2	Movie 3
User 1	-1	0	+1
User 2	-1	0	+1

new value = (old value – mean row value) [/ (max row value –min row value)]

	Movie 1	Movie 2	Movie 3	Mean
User 1	1	2	3	2
User 2	2	3	4	3

Row Normalization

- Z-score:

$$z_i = \frac{x_i - \text{mean}(x)}{\text{std}(x)}$$

$$\text{mean}(x) = \frac{1}{N} \sum_{j=1}^N x_j$$

$$\text{std}(x) = \sqrt{\frac{\sum_{j=1}^N (x_j - \text{mean}(x))^2}{N}}$$

Average “distance” from the mean
N may be N-1: population vs sample

- Measures the **number of standard deviations away from the mean**

	Movie 1	Movie 2	Movie 3
User 1	1.01	-0.87	-0.22
User 2	-1.01	0.55	0.93

	Movie 1	Movie 2	Movie 3	Mean	STD
User 1	5	2	3	3.33	1.53
User 2	1	3	4	2.66	1.53

Softmax function

- What if we want to transform the scores into a **probability distribution**, but capture the single selection of the user?
 - We want most of the probability mass to a single (or a few) restaurants
- Use the **softmax** function

$$\frac{e^{x_i}}{\sum_i e^{x_i}}$$

	Restaurant 1	Restaurant 2	Restaurant 3
User 1	0.72	0.10	0.18
User 2	0.07	0.31	0.62

	Restaurant 1	Restaurant 2	Restaurant 3
User 1	5	2	3
User 2	1	3	4

Row Normalization

- What if we want to transform the score into a **probability** that the user will visit the restaurant again
 - Different from “probability that the user will select one among the three”.
 - It is not a distribution over the restaurants, it is a distribution that over the events “will visit again”/ “will not visit again”
- One idea: **Normalize by the max score:**

	Restaurant 1	Restaurant 2	Restaurant 3
User 1	1	0.4	0.6
User 2	0.25	0.75	1

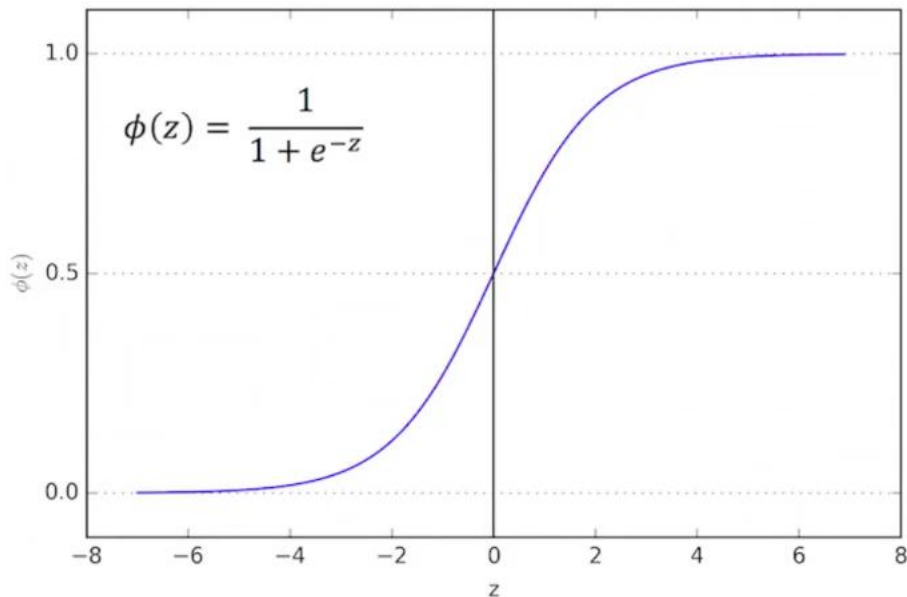
- Problem with that?
 - We have probability 1, too strong

	Restaurant 1	Restaurant 2	Restaurant 3
User 1	5	2	3
User 2	1	3	4

Logistic function

- Another idea: Use the **logistic function**:
 - Maps reals to the $[0,1]$ range
 - Mimics the step function
 - In the class of **sigmoid** functions

$$\phi(x) = \frac{1}{1 + e^{-x}}$$



	Restaurant 1	Restaurant 2	Restaurant 3
User 1	0.99	0.88	0.95
User 2	0.73	0.95	0.98

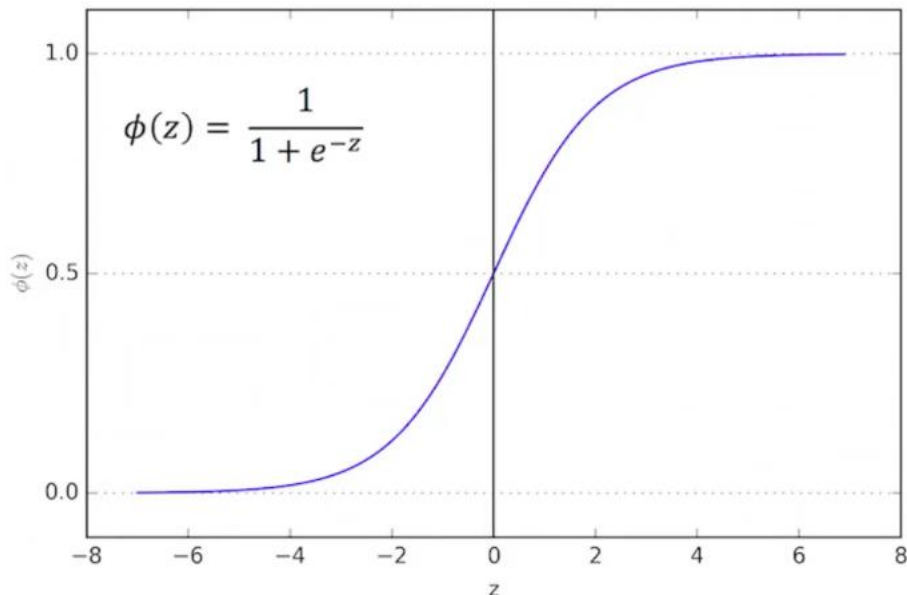
Problem: Too big values for all restaurants

	Restaurant 1	Restaurant 2	Restaurant 3
User 1	5	2	3
User 2	1	3	4

Logistic function

- Another idea: Use the **logistic function**:
 - Maps reals to the $[0,1]$ range
 - Mimics the step function
 - In the class of **sigmoid** functions

$$\phi(x) = \frac{1}{1 + e^{-x}}$$



	Restaurant 1	Restaurant 2	Restaurant 3
User 1	0.84	0.20	0.42
User 2	0.16	0.58	0.79

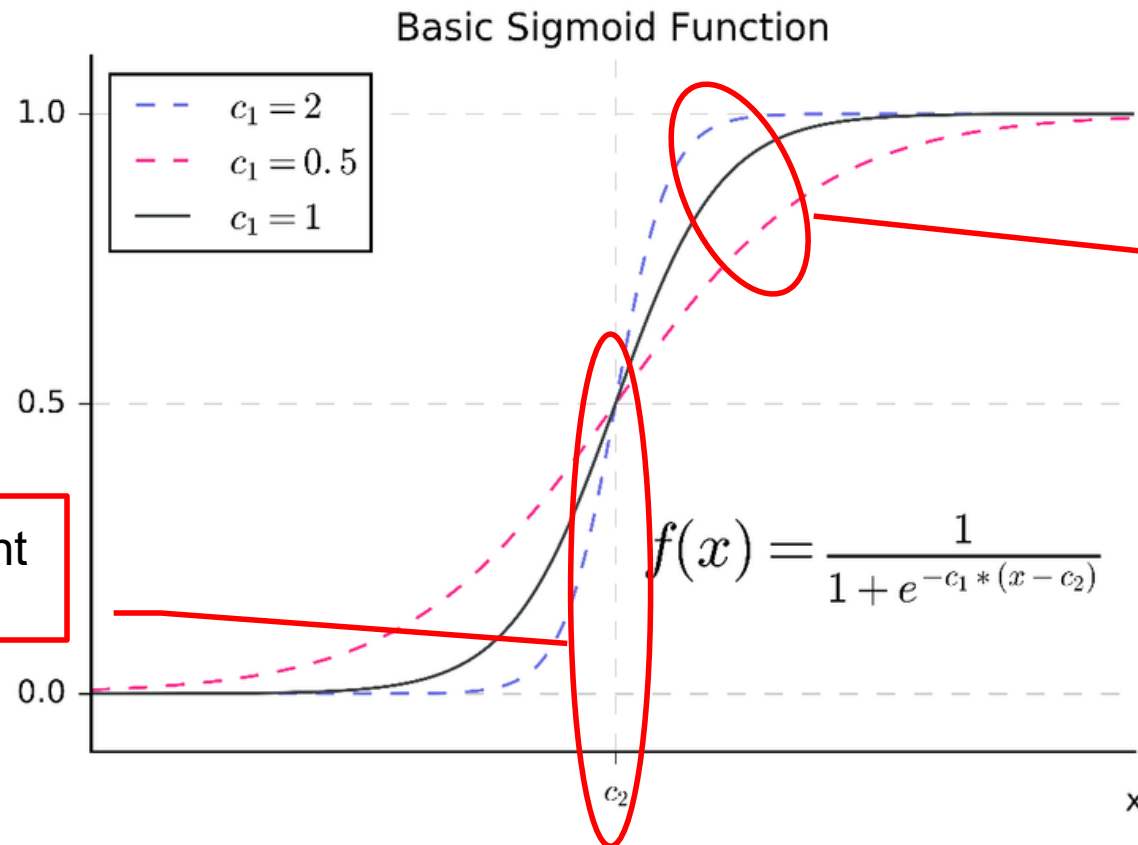
Subtract the mean
Mean value gets 50-50 probability

	Restaurant 1	Restaurant 2	Restaurant 3
User 1	1.67	-1.33	-0.33
User 2	-1.67	0.33	1.33

Sigmoid function

- General sigmoid function:
 - We can control the zero point and the slope

$$\phi(x) = \frac{1}{1 + e^{-c_1(x-c_2)}}$$



c_2 controls the 0.5 point
– change of slope

Higher c_1 closer to
a step function

Logarithm function

- Sometimes a data row/column may have a very wide range of values. Normalizing in this case will oblivate small values.

User id	Income
1	6000
2	6500
3	7000
4	4000
5	8000
6	9000
7	18000
8	36000
9	600000
10	1000000

Average = 170K

Logarithm function

- We can bring the values to the same scale by applying a logarithm to the column values.

User id	Income	Log Income
1	6000	3.778151
2	6500	3.812913
3	7000	3.845098
4	4000	3.60206
5	8000	3.90309
6	9000	3.954243
7	18000	4.255273
8	36000	4.556303
9	600000	5.778151
10	1000000	6

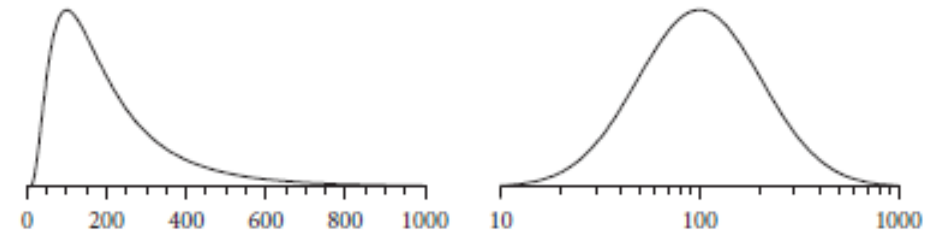


Figure 11.2. Lognormal distribution.

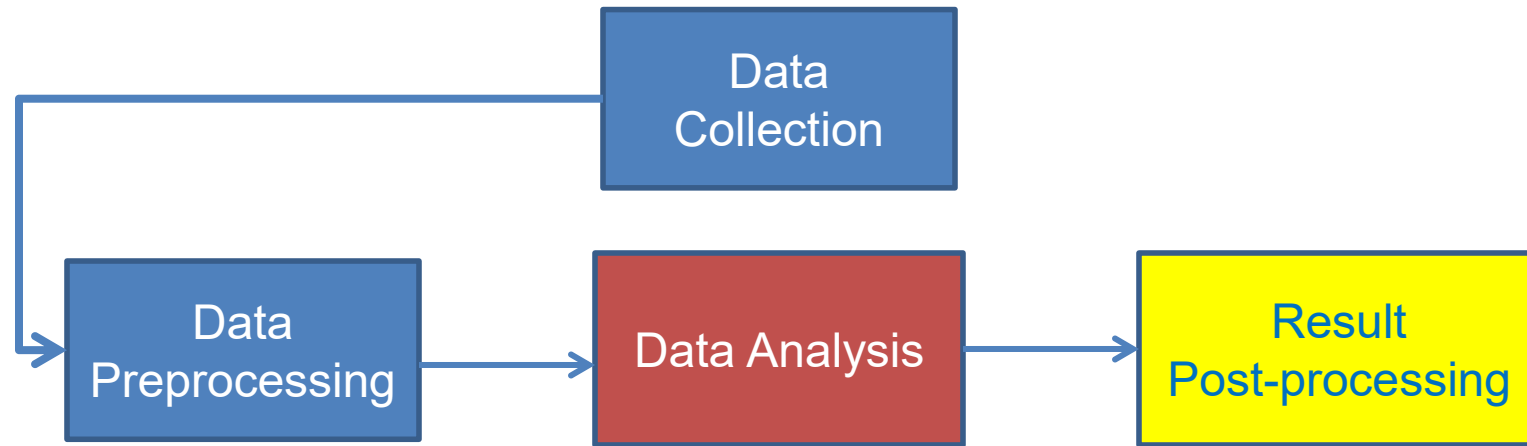
(Left) Lognormal distribution. The distribution appears Gaussian when plotted on a logarithmic axis (right) or when all values are transformed to their logarithm.

Practical advices for pre-processing

- Never delete raw data!
 - It is usually painful to collect, so raw data is precious. **Always keep a copy** so you can go back and change the processing
- **Keep the output of intermediate steps**
 - Useful for making small changes at different points in the pipeline and not run everything from scratch
- Carefully document the pre-processing steps
 - The output you will get is for the specific pre-processing you have applied.
 - For example, if you remove outliers, you lose any information they may carry
 - If you throw out some parts of the data (e.g., reviews not in English) you now have a biased sample of your data, and this bias should be documented

POST-PROCESSING

The data mining pipeline



- **Post-Processing:** Make the data actionable and useful to the user
 - Evaluation
 - Visualization
 - Statistical analysis of importance of results

Post-processing – Evaluation

- Evaluation is big part of the data mining process.
- It is part of the data analysis step, since we need to have some quality guarantees for the produced models.
- Sometimes it can also take place at post-processing time, especially if we want to evaluate a model in practice/production

Evaluation in deployment

- When a model is used and **deployed** in practice, we need to evaluate how it performs **“in vivo”**
 - If the data we worked with are representative, we expect the model to perform as **“in vitro”**
 - But real-life conditions can be unexpected.
- A/B testing: The typical approach for testing models in practice. Typically applied for online applications.
 - Some part of the traffic, chosen uniformly at random, is reserved for the test
 - Some measurable quantity – metric – is monitored.
 - The new model is compared against the existing model on this metric.
- Labeled data may need to be collected for the additional evaluation

Evaluation beyond quality

- Models are trained to optimize some metric of quality for the specific task
 - E.g., accuracy of prediction for classification, coherence for clustering.
- At post-processing time, we may want to evaluate against other metrics
- Example: Fairness and bias
 - A classification algorithm may be highly accurate but always make bad decisions for minorities.
 - We want to understand the performance of the model against these metrics
 - Go back and design models with additional requirements.
- Again, additional labeled data may be required.

Post-processing – Visualization

- The **human eye** is a powerful analytical tool
- If we visualize the data properly, we can discover patterns and demonstrate trends
- Visualization is the way to present the data so that patterns can be seen
 - E.g., histograms and plots are a form of visualization
 - There are multiple techniques (a field on its own)
- Beyond analysis, visualization is a way for a data scientist to present their work
 - Very important, otherwise the work is lost

Visualization on a map

- John Snow, London 1854

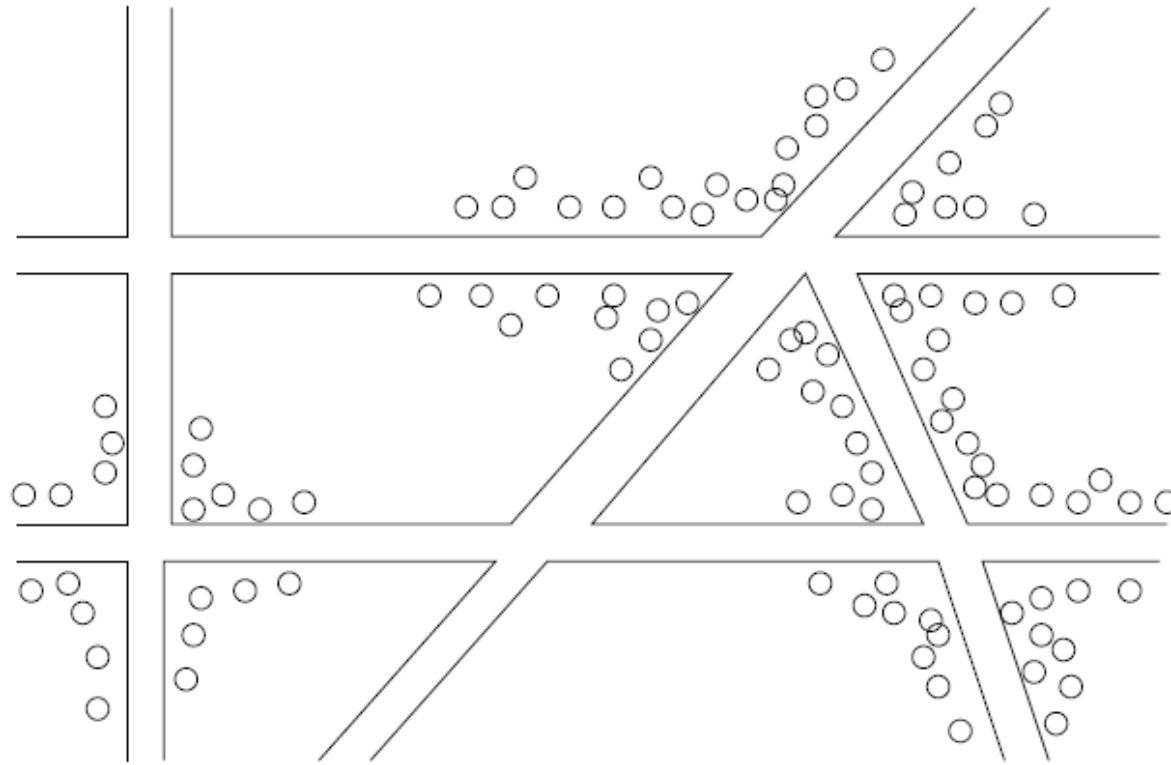


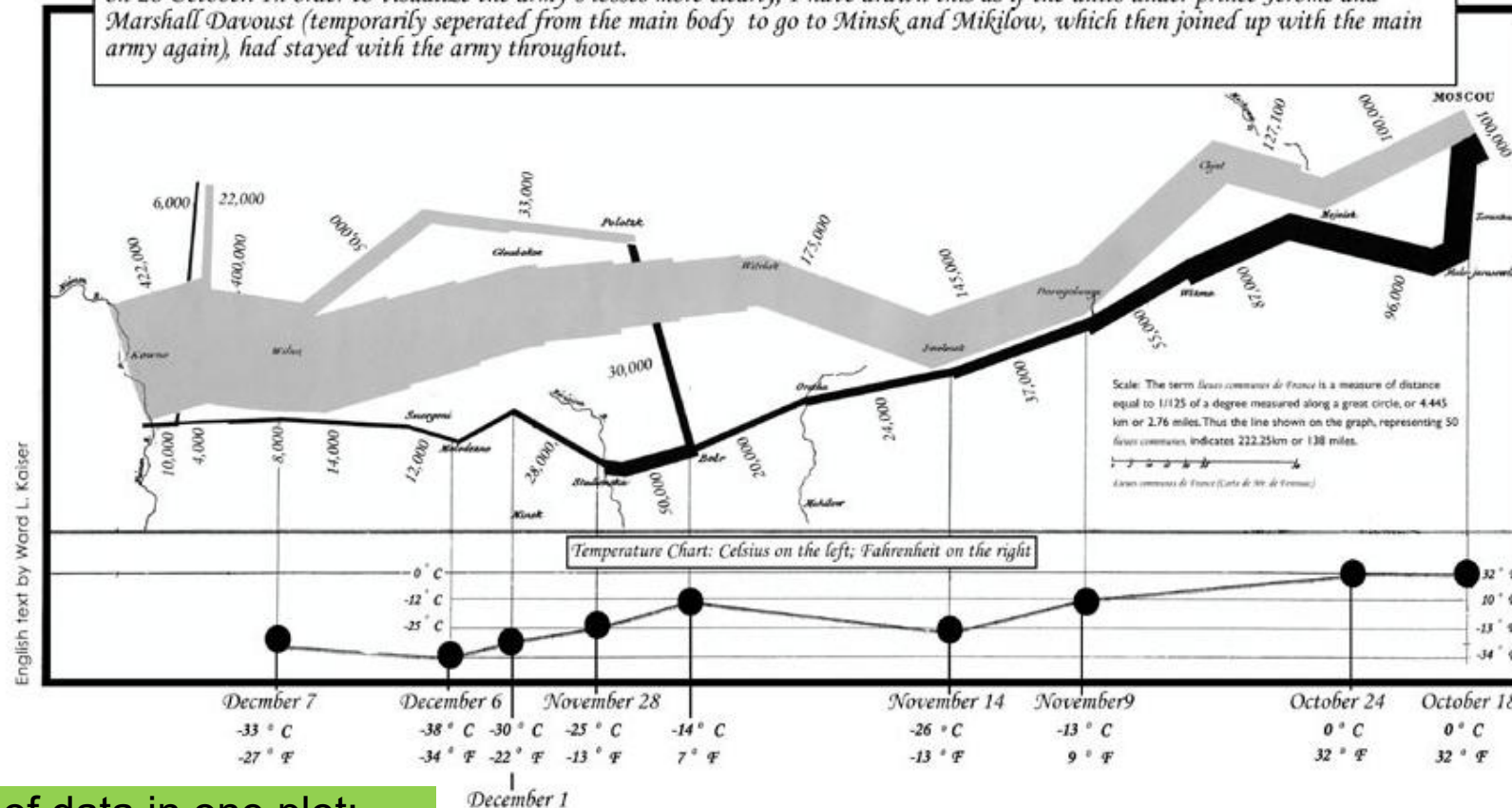
Figure 1.1: Plotting cholera cases on a map of London

Charles Minard map

Map representing the losses over time of French army troops during the Russian campaign, 1812-1813.
Constructed by Charles Joseph Minard, Inspector General of Public Works retired.

Paris, 20 November 1869

The number of men present at any given time is represented by the width of the grey line; one mm. indicates ten thousand men. Figures are also written besides the lines. Grey designates men moving into Russia; black, for those leaving. Sources for the data are the works of messrs. Thiers, Segur, Fezensac, Chambray and the unpublished diary of Jacob, who became an Army Pharmacist on 28 October. In order to visualize the army's losses more clearly, I have drawn this as if the units under prince Jerome and Marshall Davoust (temporarily separated from the main body to go to Minsk and Mikilow, which then joined up with the main army again), had stayed with the army throughout.



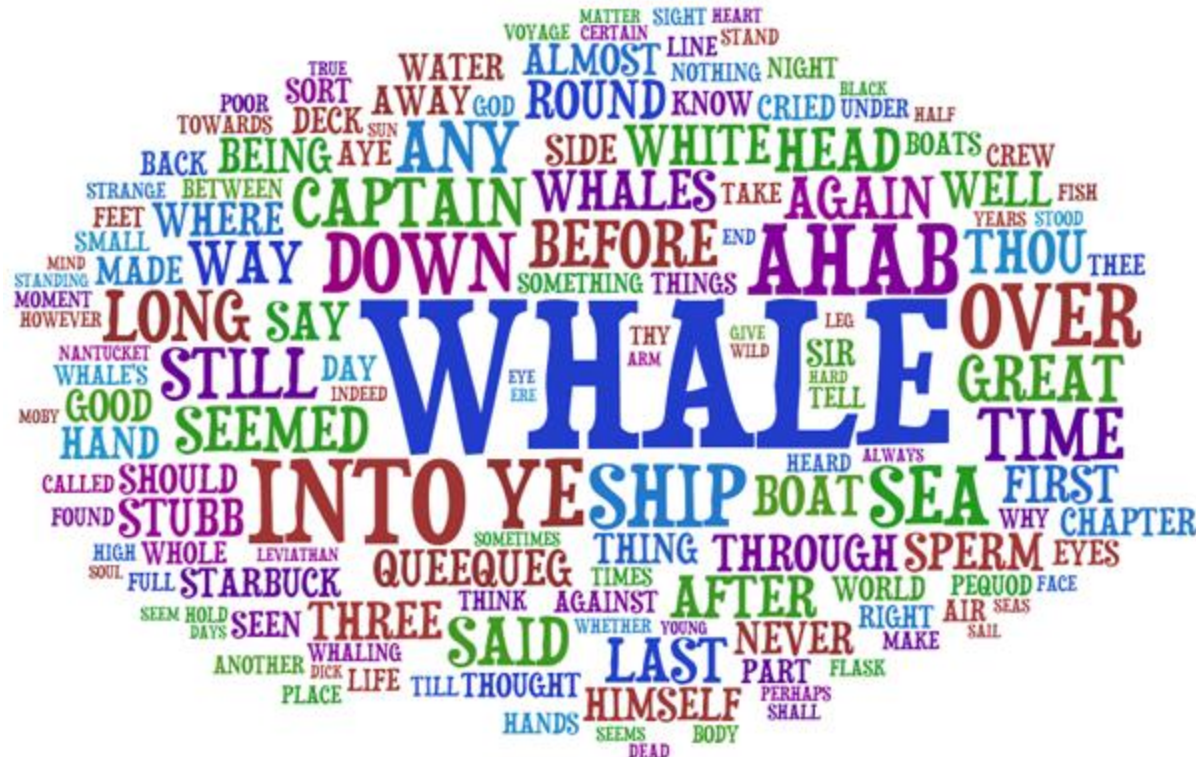
Six types of data in one plot:
size of army, temperature,
direction, location, dates etc

Another interesting visualization

- [China growth over the years](#)
- [XKCD: Earth Temperature Over Time](#)

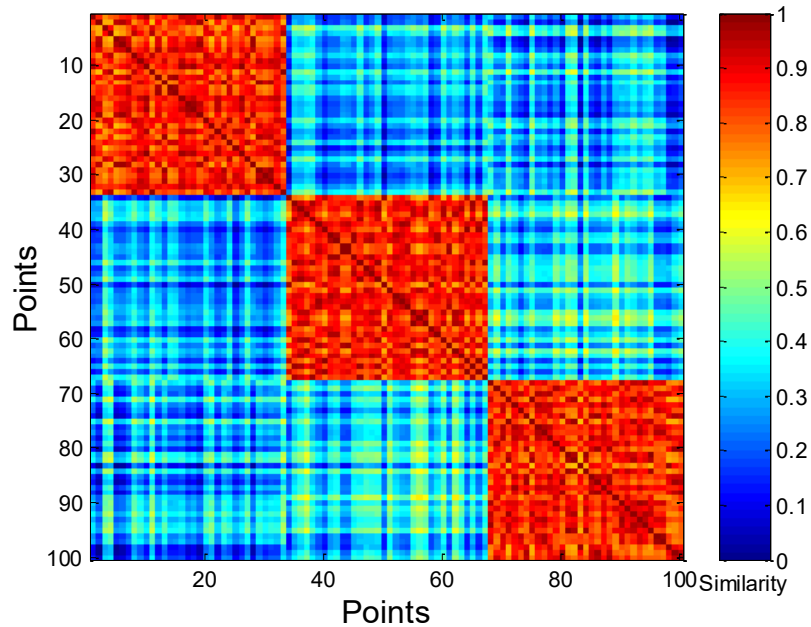
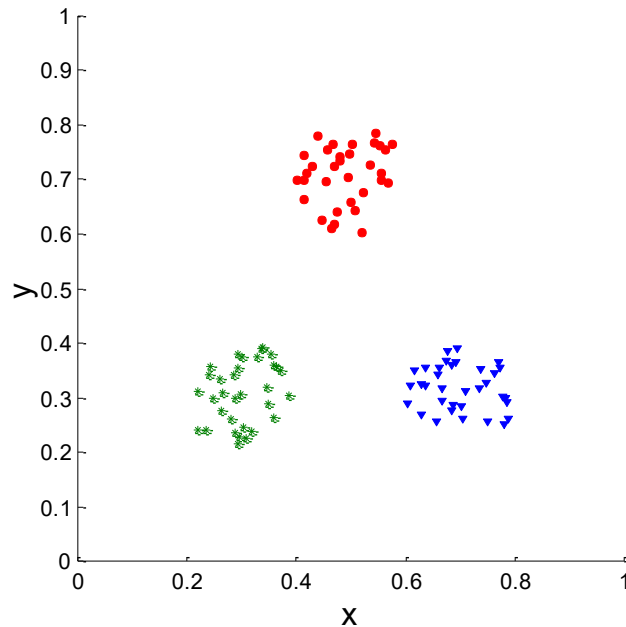
Word Clouds

- A fancy way to visualize a document or collection of documents.



Heatmaps

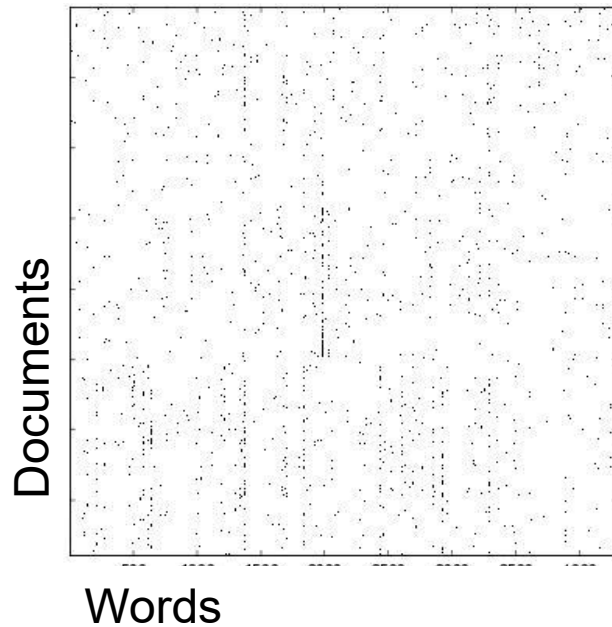
- Plot a point-to-point similarity matrix using a heatmap:
 - Deep red = high values (hot)
 - Dark blue = low values (cold)



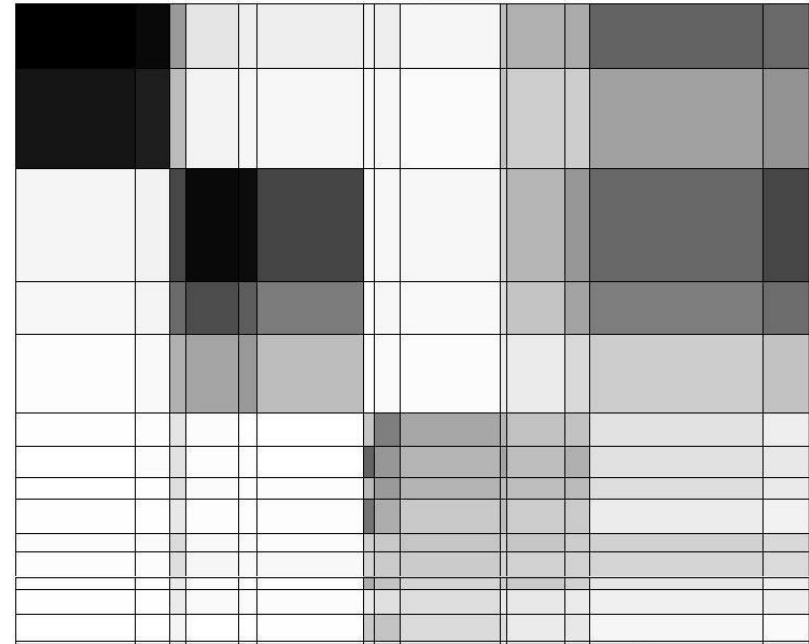
The clustering structure becomes clear in the heatmap

Heatmaps

- Heatmap (grey scale) of the data matrix
 - Document-word frequencies



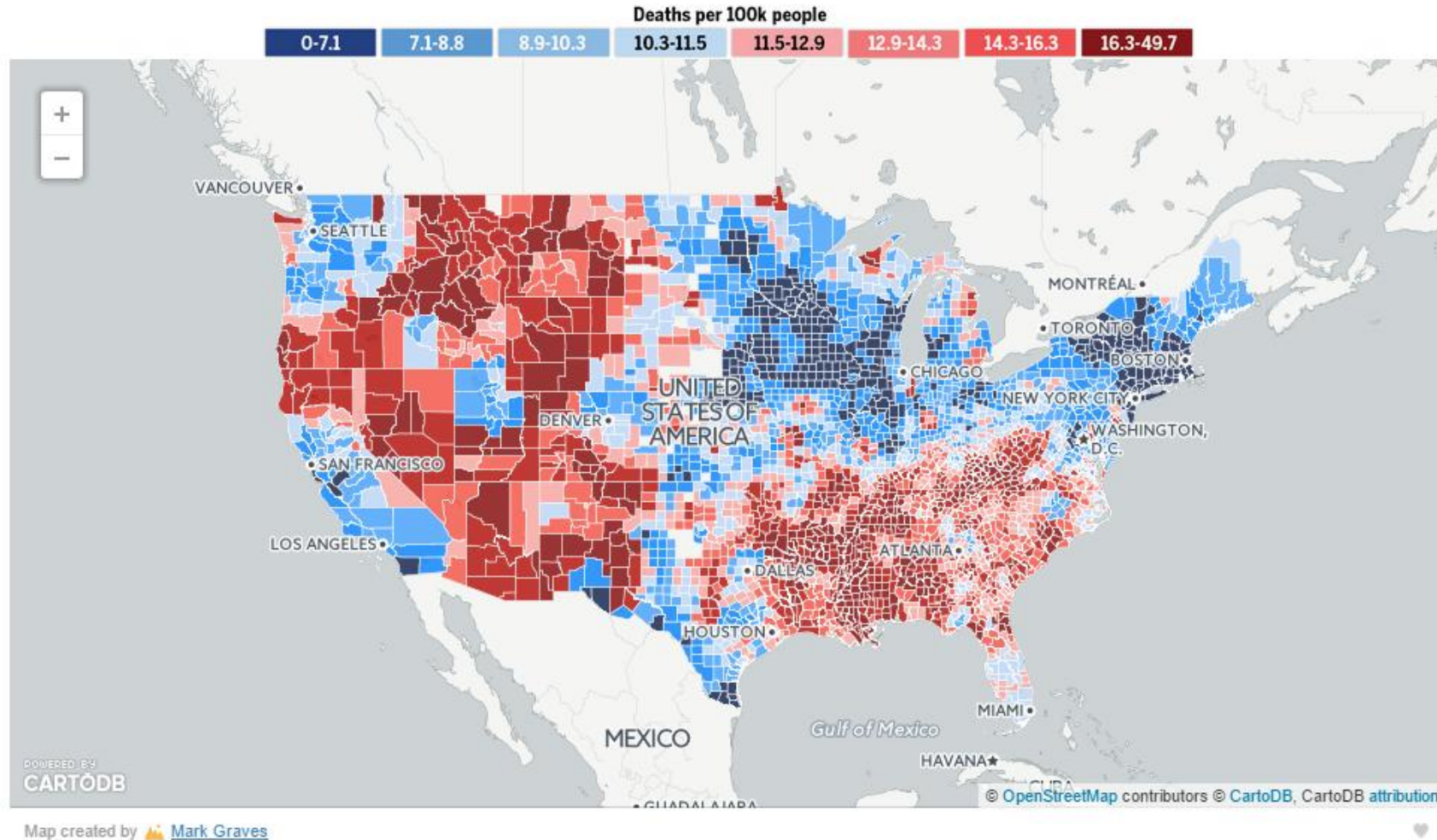
Before clustering



After clustering

Heatmaps

A very popular way to visualize spatial data



<http://projects.oregonlive.com/ucc-shooting/gun-deaths.php>

Dimensionality Reduction

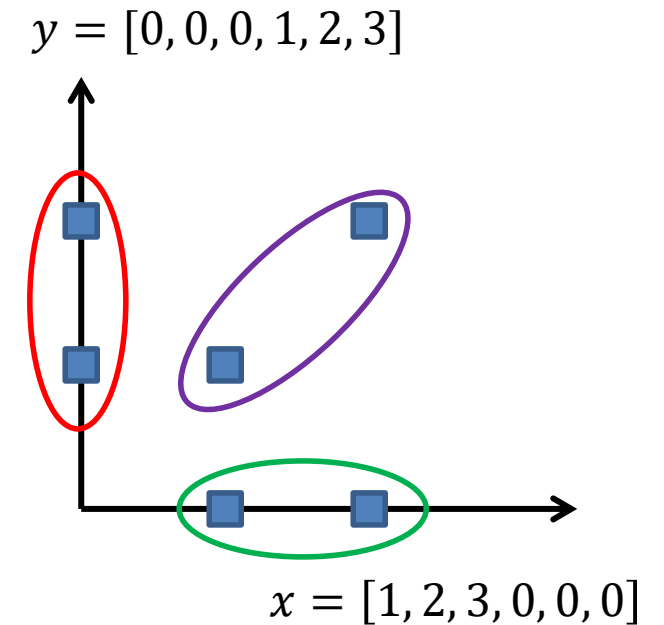
- The human eye is limited to processing visualizations in two (at most three) dimensions
- One of the great challenges in visualization is to visualize **high-dimensional data** into a **two-dimensional** space
- We can use **dimensionality reduction** for this task
- Similar to this are **distance preserving embeddings**

Example

$$D = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 & 0 \\ 2 & 4 & 6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 2 & 4 & 6 \\ 1 & 2 & 3 & 1 & 2 & 3 \\ 2 & 4 & 6 & 2 & 4 & 6 \end{bmatrix}$$

- Each row is a **multiple** of two **vectors**
 - $x = [1, 2, 3, 0, 0, 0]$
 - $y = [0, 0, 0, 1, 2, 3]$
- We can rewrite D as

$$D = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 0 & 1 \\ 0 & 2 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \end{bmatrix}$$

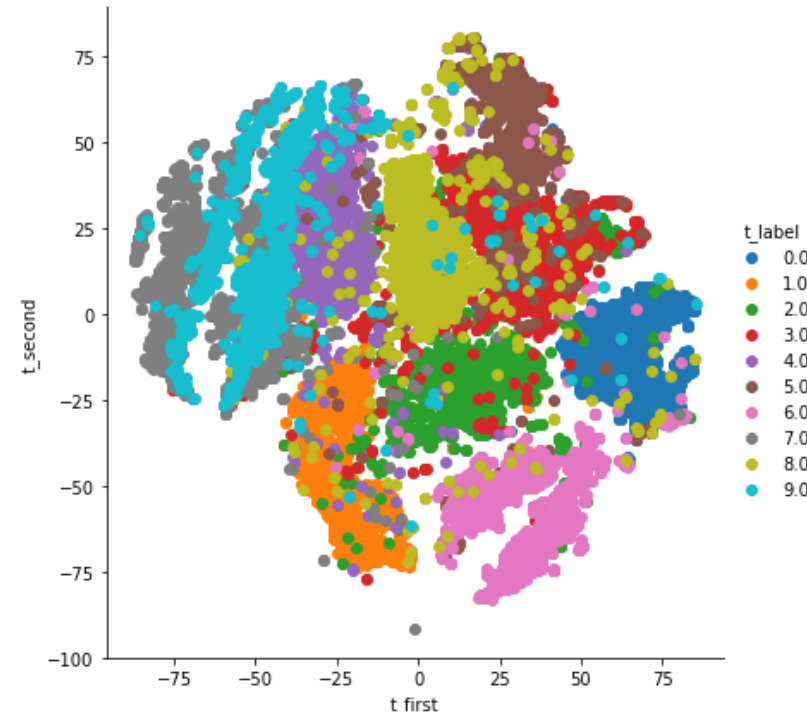
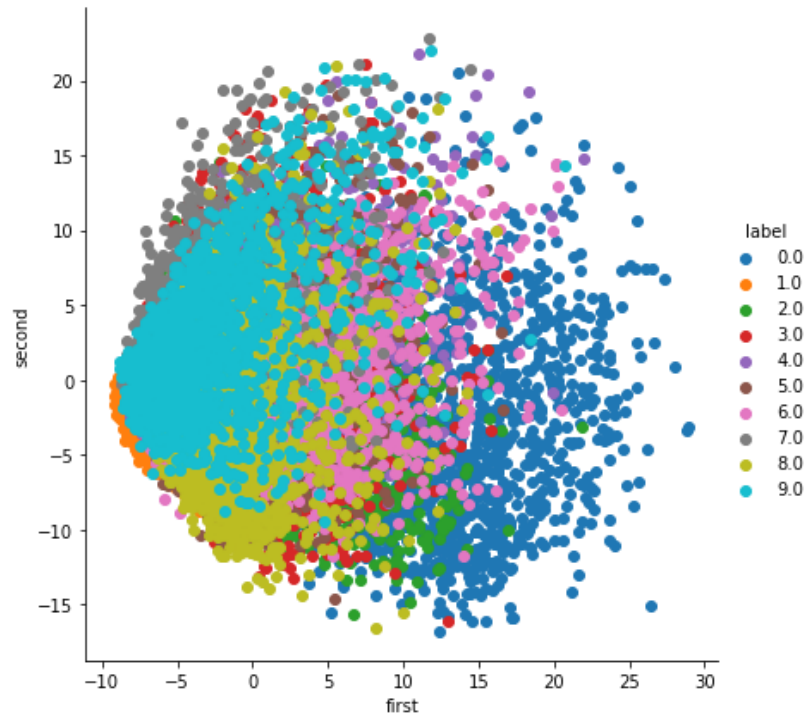


Three types of data points

Non-linear dimensionality reduction

- The idea in the previous dimensionality reduction, and visualization was to change the axes through **linear** transformations
- There are also **non-linear techniques** for dimensionality reduction that lead to better visualization
- **t-SNE** is considered the state-of-the-art

Linear
Transformation



Non-linear
Transformation

Post-processing – Statistical Significance

- When we extract knowledge from a large dataset we need to make sure that what we found is not an **artifact of randomness**
 - E.g., we find that many people buy milk and toilet paper together.
 - But many (more) people buy milk and toilet paper **independently**
- Statistical tests compare the results of an experiment with those generated by a **null hypothesis**
 - E.g., a null hypothesis is that people select items independently.
- A result is interesting if it cannot be produced by **randomness**.
 - An important problem is to define the null hypothesis correctly: What is random?

Meaningfulness of Answers

- A big data-mining risk is that you will “discover” patterns that are meaningless.
- Statisticians call it **Bonferroni's principle**: (roughly) if you look in more places for interesting patterns than your amount of data will support, you are bound to find crap.
- The **Rhine Paradox**: a great example of how **not** to conduct scientific research.

Rhine Paradox – (1)

- Joseph Rhine was a parapsychologist in the 1950's who hypothesized that some people had Extra-Sensory Perception.
- He devised (something like) an experiment where subjects were asked to guess 10 hidden cards – red or blue.
- He discovered that almost 1 in 1000 had ESP – they were able to get all 10 right!

Rhine Paradox – (2)

- He told these people they had ESP and called them in for another test of the same type.
- Alas, he discovered that almost all of them had lost their ESP.
 - Why?
- What did he conclude?
 - Answer on next slide.

Rhine Paradox – (3)

- He concluded that you shouldn't tell people they have ESP; it causes them to lose it.